

SOCIAL NETWORK ANALYSIS

Network Analysis (Round 1)

Project Report By

Ronit Gupta(21UCS173)

Ruthvik Devavarapu (21UCS174)

Saumya Saraswat(21UCS184)

Saunak Kushwaha(21UCS185)

Course Instructor

Dr. Sakthi Balan

Table Of Contents

Sr. No.	Topic	Page
1	Problem Statement	2
2	Tools Used	2
3	Network 1 - Euroroads	4
	a) Overview	4
	b) Network Statistics	5
	c) Centrality Measures	6
	d) Clustering Coefficients	22
	e) Transitivity and Reciprocity	24
	f) Largest Connected Component	24
	g) Complete Network Visualization	26
4	Network 2 - General Relativity and Quantum Cosmology Collaboration Network	27
	a) Overview	27
	b) Network Statistics	27
	c) Centrality Measures	29
	d) Clustering Coefficients	41
	e) Transitivity and Reciprocity	42
	f) Largest Connected Component	43
	g) Complete Network Visualization	44
5	Comparative Analysis Between The Two Networks	45

Problem Statement

You can choose two datasets (with at least 1000 number of nodes in the network) ONLY from the given links.

Once you fix the datasets, you have to do the following:

- Summarise the network statistics - Plot the degree distribution of the network, find the max and minimum degree, average degree, and Standard deviation of the degree distribution. (Visualisation wherever necessary)
 - Find all centrality measures studied in the class - Degree centrality, Eigenvector centrality, Katz and PageRank centrality, clustering coefficients (both local and global), find average local clustering coefficient and compare with global clustering coefficient, find the betweenness and closeness centrality and reciprocity and transitivity that we have studied in the class using appropriate algorithms (you may use specific packages for this or write your algorithm for the same).
 - Produce a project report with all details like the:
 - Complete Description of the Data Set - What is the network about - what are nodes and edges depicting in the network?
 - In the dataset, what are the centrality measures you studied (with the top 10 as a table and the visualisation for the complete network)
 - What are the clustering coefficients (local, global) for this data set (with the top 10 as a table and the visualisation for the complete network)
 - Which are all (centrality measures) appropriate for your study?
 - What is the output of all centrality measures? Visualise it graphically. Do not put the value in text format! (no marks if the output is given ONLY in text format).
 - What are your inferences from the output?
 - Compare and contrast the two networks taken according to the centrality measures study and present your inferences from it.
-

Tools Used

```
#Libraries for Data Manipulation
import numpy as np
import pandas as pd

#Libraries for Graph Plotting
import seaborn as sns
import matplotlib.pyplot as plt
import matplotlib.colors as mcolors

#Libraries for Network Analysis
import networkx as nx
```

Besides using some standard Data Manipulation and Visualization packages, we have used **NetworkX**, a Python package, to create, manipulate, and study complex networks' structure, dynamics, and functions to perform the required network analysis in our project.

About NetworkX

NetworkX is a Python package for the creation, manipulation, and study of the structure, dynamics, and functions of complex networks. It provides:

- Tools for the study of the structure and dynamics of social, biological, and infrastructure networks;
- a standard programming interface and graph implementation that is suitable for many applications;
- a rapid development environment for collaborative, multidisciplinary projects;
- an interface to existing numerical algorithms and code written in C, C++, and FORTRAN; and
- The ability to work painlessly with large nonstandard data sets.

With NetworkX, you can load and store networks in standard and nonstandard data formats, generate many types of random and classic networks, analyse network structure, build network models, design new network algorithms, draw networks, and more.

Network 1 - Euroroads

Overview

This is the international E-road network located primarily in Europe. The network is undirected; nodes represent cities, and an edge between two nodes indicates that an E-road connects them.

- The network we get from the dataset contains nodes representing the cities, and the edges represent the roads between the cities.

Additional Statistics

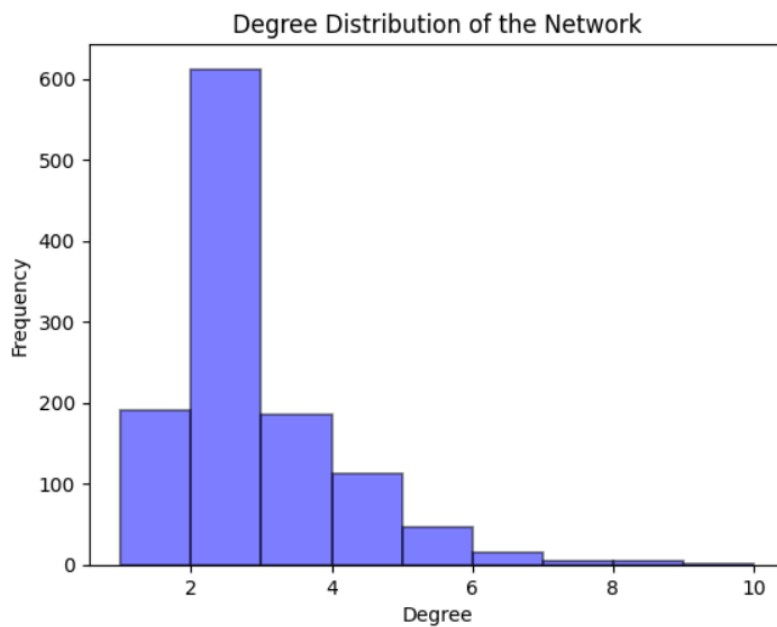
Nodes	1,174
Edges	1,417
Loop count	0
Triangle count	32
Maximum degree	10
Average degree	2.413 97
Fill	0.002 057 94
Size of LCC	1,039
Diameter	62
90-Percentile effective diameter	33.340 5
Mean distance	19.181 2
Power law exponent	2.289 90
Clustering coefficient	0.033 886 3
Algebraic connectivity	0.001 160 30

Network Statistics

Degree Distribution Of This Network

We have used the code below to obtain a degree distribution for our network.

```
degrees = [G.degree(node) for node in G.nodes()]
plt.hist(degrees, bins=range(min(degrees), max(degrees)+1), alpha=0.5, color='b', edgecolor='black', linewidth=1.2)
plt.xlabel('Degree')
plt.ylabel('Frequency')
plt.title('Degree Distribution of the Network')
plt.show()
```



Additional Statistics

The image below presents our code and the obtained results for additional statistics for our chosen network. Besides the number of nodes and edges, we have also found out the following:

- Maximum Degree of a node(city name) in the network.
- Minimum Degree of a node(city name) in the network.
- The Average Degree of a node in the network.
- The Standard deviation of the Degree Distribution in the network.

```

#Calculating the Degree of Each Node In the Network
degrees = dict(G.degree())

#Find the nodes with maximum and minimum degree
max_degree_node = max(degrees, key=degrees.get)
min_degree_node = min(degrees, key=degrees.get)

#Adjust node numbering and mapping
node_name_mapping = {str(i+1): name for i, name in enumerate(node_names)} # Adjusted for node numbers
nx.relabel_nodes(G, node_name_mapping, copy=False)

#Print the name of the city with the maximum and minimum degree
print("City with maximum degree:", max_degree_node, "Degree:", degrees[max_degree_node])
print("City with minimum degree:", min_degree_node, "Degree:", degrees[min_degree_node])

```

```

City with maximum degree: Moscow Degree: 10
City with minimum degree: Greenock Degree: 1

```

```

print('Number of nodes', len(G.nodes))
print('Number of edges', len(G.edges))
avg_degree = np.mean(degrees)
std_dev = np.std(degrees)

print(f"Standard Deviation: {std_dev}")
print(f"Average Degree: {avg_degree}")

```

```

Number of nodes 1174
Number of edges 1417
Standard Deviation: 1.188678457809932
Average Degree: 2.41396933560477

```

Centrality Measures

Degree Centrality

Degree centrality in a graph represents the number of edges that are incident upon a node (i.e., the number of connections a node has). In simpler terms, it measures how connected a node is to other nodes in the graph.

In the context of a road network graph, degree centrality would indicate the number of roads that converge at a specific city or intersection. Cities with a high degree of centrality serve as major transportation hubs, where numerous roads intersect, making them pivotal points for travel and connectivity. On the other hand, cities with a low degree of centrality have fewer roads converging at them, indicating a lower level of connectivity and importance in the network.

```
def degree centrality(G, node):
    neighbors = G[node]
    degree = len(neighbors)
    return degree / (len(G) - 1)

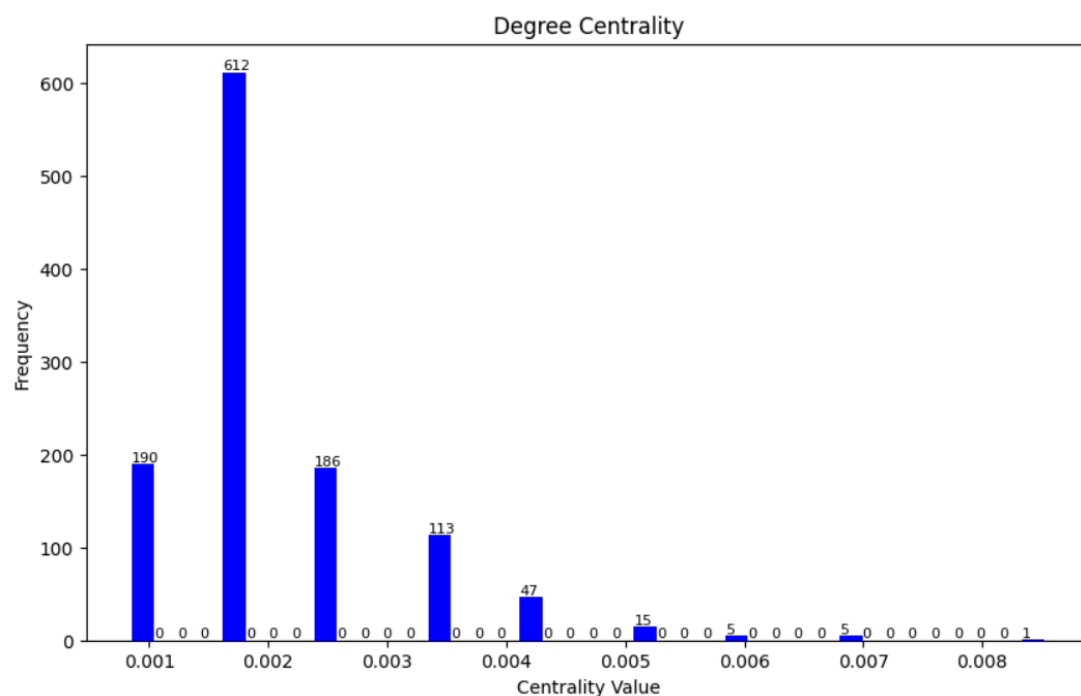
# Calculate degree centrality for each node
deg_centrality = {}
for node in G.nodes():
    deg_centrality[node] = degree_centrality(G, node)

# Print each node and its Degree Centrality
for node, centrality in deg_centrality.items():
    print(f"{node}: Degree Centrality = {centrality}")
```

```
# Plot distribution of centrality values for Degree Centrality
plt.figure(figsize=(10, 6))
plt.hist(list(deg_centrality.values()), bins=40, color='b', alpha=0.7) # Increase bin size to 40
plt.title("Degree Centrality")
plt.xlabel('Centrality Value')
plt.ylabel('Frequency')

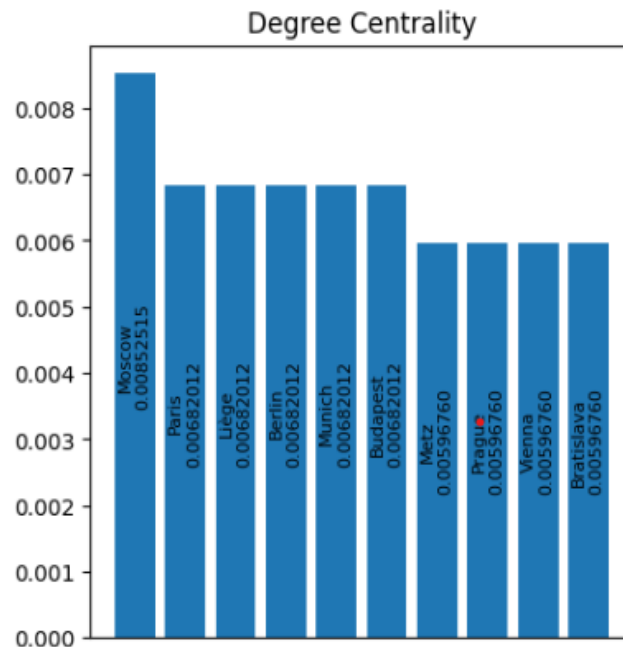
# Write frequency on top of each bin
for i in range(40):
    plt.text(plt.hist(list(deg_centrality.values()), bins=40, color='b', alpha=0.7)[1][i],
             plt.hist(list(deg_centrality.values()), bins=40, color='b', alpha=0.7)[0][i],
             str(int(plt.hist(list(deg_centrality.values()), bins=40, color='b', alpha=0.7)[0][i])),
             fontsize=8, ha='left', va='bottom')

plt.show()
```



Top 10 Description

Node	Degree Centrality
Moscow	0.008525
Paris	0.006820
Liege	0.006820
Berlin	0.006820
Munich	0.006820
Budapest	0.006820
Metz	0.005968
Prague	0.005968
Vienna	0.005968
Bratislava	0.005968



Eigenvector Centrality

Eigenvector centrality assigns relative scores to all nodes in the network based on the concept that connections to high-scoring nodes contribute more to the score of the node in question. By calculating the eigenvector centrality of each city, we can identify the most influential cities in the network, which can serve as essential junctions or hubs for transportation and communication between other influential cities.

For example, if a city has a high eigenvector centrality score, it means that it is connected to other cities that also have high centrality scores, indicating that it is a hub for transportation and communication in the network. This information can help plan transportation routes, identify potential bottlenecks in the network, and optimise resource allocation.

Furthermore, eigenvector centrality can also help identify communities or clusters of cities within the network, which can be useful for regional planning and development. By analyzing the eigenvector centrality scores of cities within a region, we can identify key transportation hubs and plan infrastructure development accordingly.

```
#Calculate Eigenvector Centrality
eigenvector_centrality = nx.eigenvector_centrality(G)

#Print each node and its Eigenvector Centrality
for node, centrality in eigenvector_centrality.items():
    print(f"Node {node}: Eigenvector Centrality = {centrality}")
```

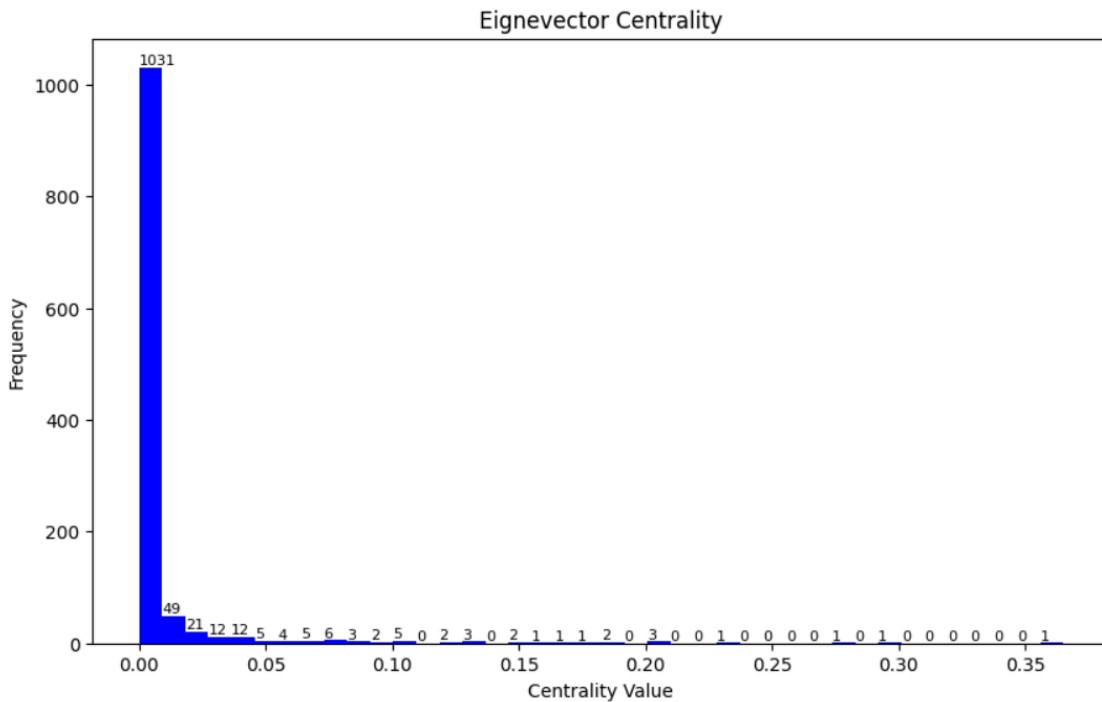
```

# Plot distribution of centrality values for Eigenvector Centrality
plt.figure(figsize=(10, 6))
plt.hist(list(eigenvector_centrality.values()), bins=40, color='b', alpha=0.7) # Increase bin size to 40
plt.title("Eigenvector Centrality")
plt.xlabel('Centrality Value')
plt.ylabel('Frequency')

# Write frequency on top of each bin
for i in range(40):
    plt.text(plt.hist(list(eigenvector_centrality.values()), bins=40, color='b', alpha=0.7)[1][i],
             plt.hist(list(eigenvector_centrality.values()), bins=40, color='b', alpha=0.7)[0][i],
             str(int(plt.hist(list(eigenvector_centrality.values()), bins=40, color='b', alpha=0.7)[0][i])),
             fontsize=8, ha='left', va='bottom')

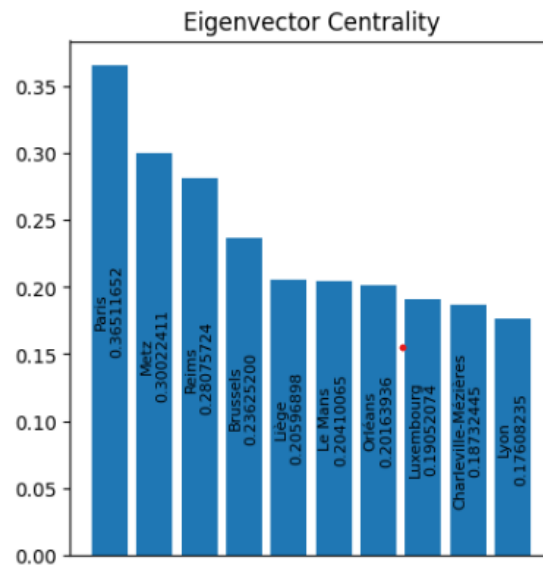
plt.show()

```



Top 10 Description

Nodes	Eigenvector Centrality
Paris	0.365117
Metz	0.300224
Reims	0.280757
Brussels	0.236252
Liege	0.205969
Le Mans	0.204101
Orleans	0.201639
Luxembourg	0.190521
Charleville-Mezieres	0.187324
Lyon	0.176082



Katz Centrality

Katz centrality quantifies the influence of a node based on its immediate neighbours and the neighbours of its neighbours, and so on, with diminishing weights for nodes that are farther away.

Cities with higher Katz centrality scores are crucial hubs in a transportation network that is well-connected to other cities and linked to well-connected cities. This centrality measure considers direct and indirect connections, favouring cities with strong ties to essential locations. It identifies key bridges and central points, enhancing network efficiency. Analyzing Katz centrality helps pinpoint major transportation hubs, ensuring smoother travel and network accessibility, which is vital for overall road network functionality and efficiency.

```

def katz(G, alpha=0.1, beta=1.0, normalized=True, weight=None):

    if len(G) == 0:
        return {}
    try:
        nodelist = beta.keys()
        if set(nodelist) != set(G):
            raise nx.NetworkXError("beta dictionary must have a value for every node")
        b = np.array(list(beta.values()), dtype=float)
    except AttributeError:
        nodelist = list(G)
        try:
            b = np.ones((len(nodelist), 1)) * beta
        except (TypeError, ValueError, AttributeError) as err:
            raise nx.NetworkXError("beta must be a number") from err

    A = nx.adjacency_matrix(G, nodelist=nodelist, weight=weight).todense().T
    n = A.shape[0]
    centrality = np.linalg.solve(np.eye(n, n) - (alpha * A), b).squeeze()

    # Normalize: rely on truediv to cast to float, then tolist to make Python numbers
    norm = np.sign(sum(centrality)) * np.linalg.norm(centrality) if normalized else 1
    return dict(zip(nodelist, (centrality / norm).tolist()))

katz_centrality = katz(G, alpha=0.1, beta=1.0, normalized=True, weight=None)

# Print each node and its Katz Centrality
for node, centrality in katz_centrality.items():
    print(f"Node {node}: Katz Centrality = {centrality}")

```

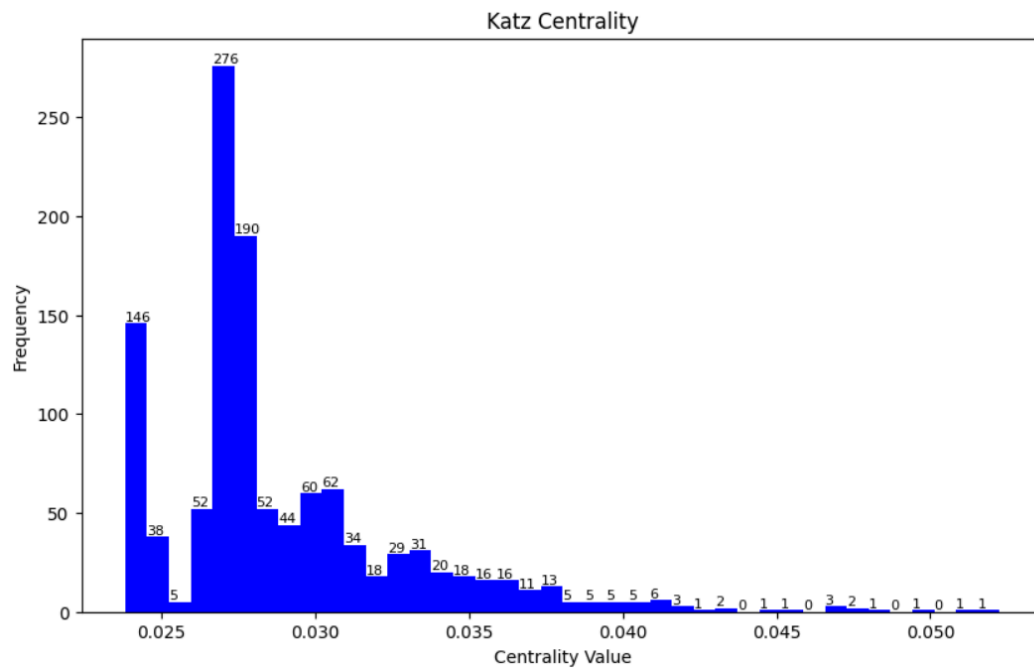
Parameter Description

1. **alpha:** Attenuation factor for the Katz centrality computation. It controls the impact of indirect paths on centrality. A smaller alpha reduces the influence of distant nodes, while a more significant alpha gives more weight to longer paths.
2. **beta:** A parameter that allows you to add a constant to each node's centrality, which can help control centrality values. It can help adjust centrality scores to fit specific requirements or to balance the influence of different factors.
3. **normalized:** If True, the Katz centrality scores are normalized to sum up to 1. Normalization can be helpful when comparing centrality scores across different networks or when interpreting the scores in a relative manner.
4. **weight:** None, string, or dictionary. If None, all edge weights are considered equal (1). If a string, this should be the key for edge weight in the edge data dictionary. If a dictionary, keys are node pairs and values are the edge weight between nodes.

```
# Plot distribution of centrality values for Katz Centrality
plt.figure(figsize=(10, 6))
plt.hist(list(katz_centrality.values()), bins=40, color='b', alpha=0.7) # Increase bin size to 40
plt.title("Katz Centrality")
plt.xlabel('Centrality Value')
plt.ylabel('Frequency')

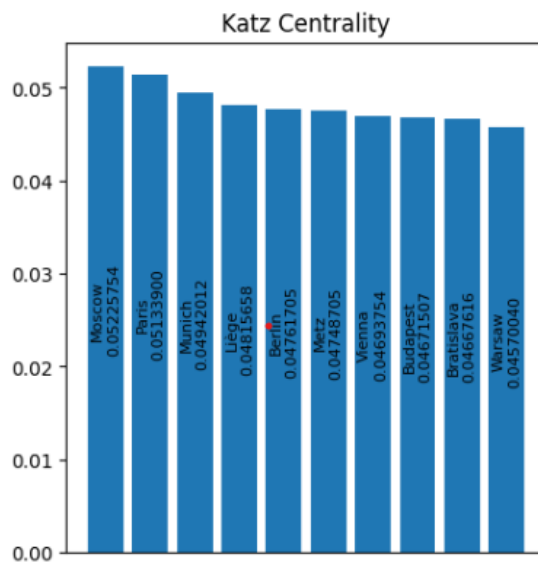
# Write frequency on top of each bin
for i in range(40):
    plt.text(plt.hist(list(katz_centrality.values()), bins=40, color='b', alpha=0.7)[1][i],
             plt.hist(list(katz_centrality.values()), bins=40, color='b', alpha=0.7)[0][i],
             str(int(plt.hist(list(katz_centrality.values()), bins=40, color='b', alpha=0.7)[0][i])),
             fontsize=8, ha='left', va='bottom')

plt.show()
```



Top 10 Description

Node	Katz Centrality
Moscow	0.052258
Paris	0.051339
Munich	0.049420
Liege	0.048157
Berlin	0.047617
Metz	0.047487
Vienna	0.046938
Budapest	0.046715
Bratislava	0.046676
Warsaw	0.045700



PageRank Centrality

The PageRank of a network measures the importance or centrality of a node within the network. It is calculated based on the number and quality of connections between nodes.

The PageRank measure helps understand a transportation network's overall structure and functionality. It can help identify key transportation hubs and bottlenecks and potential areas for improvement in connectivity and accessibility. By understanding the relative importance of different cities in the network, transportation planners can make more informed decisions about where to allocate resources and how to improve the overall efficiency of the transportation system.

```
# Calculate Page Rank

#alpha: Damping parameter for PageRank, representing the probability of following a link on the graph vs. jumping to a random node.
#       A higher alpha means more weight is given to following links, while a lower alpha gives more weight to jumping to random nodes.
#       The default value of 0.8 is commonly used and provides a good balance between these two factors.

def page_rank(G, alpha=0.85, max_iter=100, tol=1.0e-6, nstart=None, weight='weight'):

    if G.is_directed():
        in_degree = dict(nx.in_degree(G, weight=weight))
        min_in_degree = min(in_degree.values())
        if min_in_degree == 0:
            raise nx.NetworkXError("Graph contains nodes with no incoming edges.")
    else:
        in_degree = dict(nx.degree(G, weight=weight))

    if nstart is None:
        r = {node: 1/len(G) for node in G}
    else:
        r = nstart

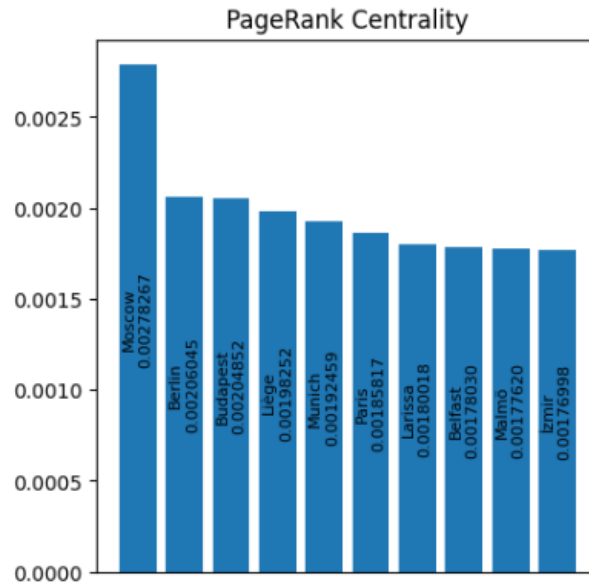
    for _ in range(max_iter):
        rlast = r.copy()
        # Power iteration
        for n in G:
            r[n] = (1 - alpha) / len(G) + alpha * sum(rlast[m] / in_degree[m] for m in G.neighbors(n) if m in rlast)
        # check for convergence
        err = max(abs(r[n] - rlast[n]) for n in r)
        if err < tol:
            return r

    return r

# Calculate Page Rank for each node
pr = page_rank(G, alpha=0.8)

# Print each node and its Page Rank
for node, centrality in pr.items():
    print(f"Node {node}: Page Rank = {centrality}")
```


Node	Page Rank Centrality
Moscow	0.002783
Berlin	0.002060
Budapest	0.002049
Liege	0.001983
Munich	0.001925
Paris	0.001858
Larissa	0.001800
Belfast	0.001780
Malmo	0.001776
Izmir	0.001770



Betweenness Centrality

Betweenness centrality in a graph represents the extent to which a node lies on the shortest paths between other nodes. Specifically, it measures the number of times a node acts as a bridge along the shortest path between pairs of other nodes in the graph.

In the context of a road network represented as a graph, betweenness centrality is a measure that identifies the importance of a node based on the number of shortest paths that pass through it. Nodes with high betweenness centrality act as critical connectors within the network, as they lie on many of the shortest paths between pairs of nodes. These nodes facilitate efficient traffic flow and connectivity within the road network.

Identifying nodes with high betweenness centrality is crucial for various transportation planning and management aspects. Firstly, these nodes are key locations for transportation infrastructure planning because they represent strategic points where significant traffic converges. By focusing on these nodes, urban planners and engineers can prioritize investments in infrastructure development to enhance connectivity and alleviate congestion in the network.

```
#Calculate Betweenness Centrality

#normalized: If True, the betweenness centrality scores are normalized by dividing by the maximum possible betweenness centrality.
#             Normalization is useful for comparing centrality scores across graphs of different sizes.

#endpoints: If False, only considers paths between nodes as they are internal to the node set.
#            If True, considers both endpoints of the paths as adjacent to other nodes outside the node set.
#            Setting endpoints to False is often appropriate when analyzing networks where nodes represent entities that cannot pass through each other.

bet Centrality = nx.betweenness Centrality(G, normalized = True, endpoints = False)

#Print each node and its Betweenness Centrality (excluding nodes with centrality = 0)
for node, centrality in bet Centrality.items():
    if centrality != 0:
        print(f"Node {node}: Betweenness Centrality = {centrality}")
```



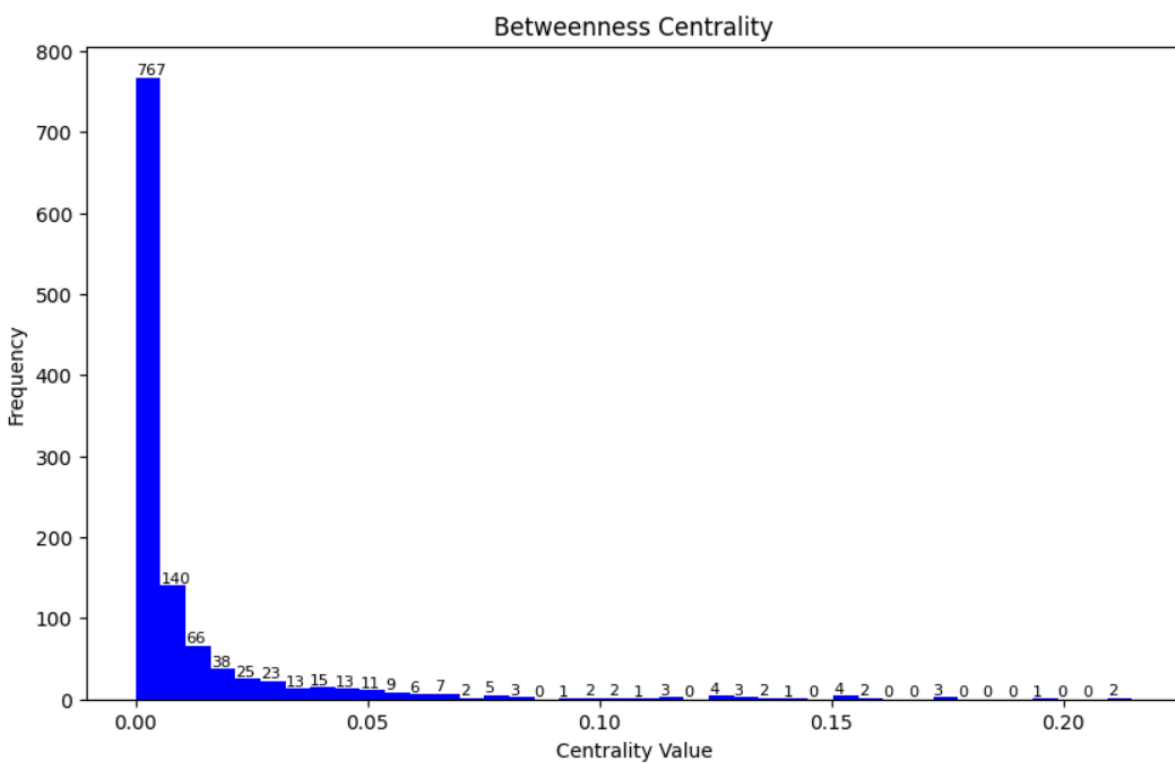
```

# Plot distribution of centrality values for Betweenness Centrality
plt.figure(figsize=(10, 6))
plt.hist(list(bet_centrality.values()), bins=40, color='b', alpha=0.7) # Increase bin size to 40
plt.title("Betweenness Centrality")
plt.xlabel('Centrality Value')
plt.ylabel('Frequency')

# Write frequency on top of each bin
for i in range(40):
    plt.text(plt.hist(list(bet_centrality.values()), bins=40, color='b', alpha=0.7)[1][i],
             plt.hist(list(bet_centrality.values()), bins=40, color='b', alpha=0.7)[0][i],
             str(int(plt.hist(list(bet_centrality.values()), bins=40, color='b', alpha=0.7)[0][i])),
             fontsize=8, ha='left', va='bottom')

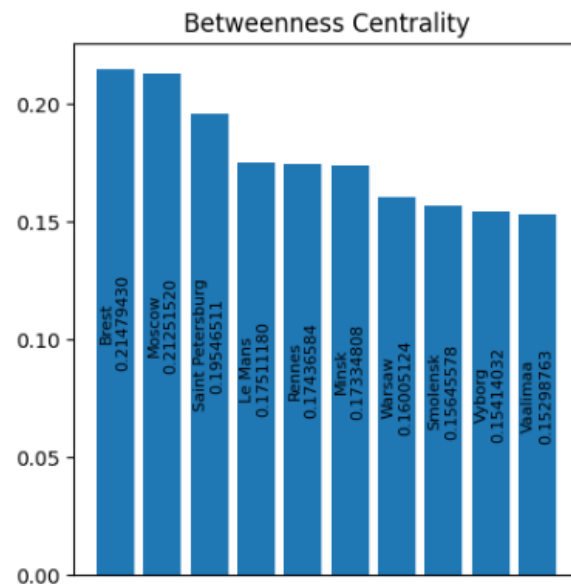
plt.show()

```



Top 10 Description

Nodes	Betweenness Centrality
Brest	0.214794
Moscow	0.212515
Saint Petersburg	0.195465
Le Mans	0.175112
Rennes	0.174366
Minsk	0.173348
Warsaw	0.160051
Smolensk	0.156456
Vyborg	0.154140
Vaalimaa	0.152988



Closeness Centrality

Closeness centrality in a graph represents how close a node is to all other nodes. Specifically, it measures the average shortest path length from a node to all other nodes in the graph.

In a road network, cities with higher closeness centrality are typically more centrally located and have shorter average distances to reach other cities. This means they are potentially more important or influential regarding transportation and connectivity. For example, a city with a high closeness centrality score could quickly reach other cities in the network, making it an ideal location for transportation hubs or distribution centres.

```
#Calculate Closeness centrality
close_centrality = nx.closeness_centrality(G)

#Print each node and its Closeness centrality
for node, centrality in close_centrality.items():
    print(f"Node {node}: Closeness centrality = {centrality}")
```

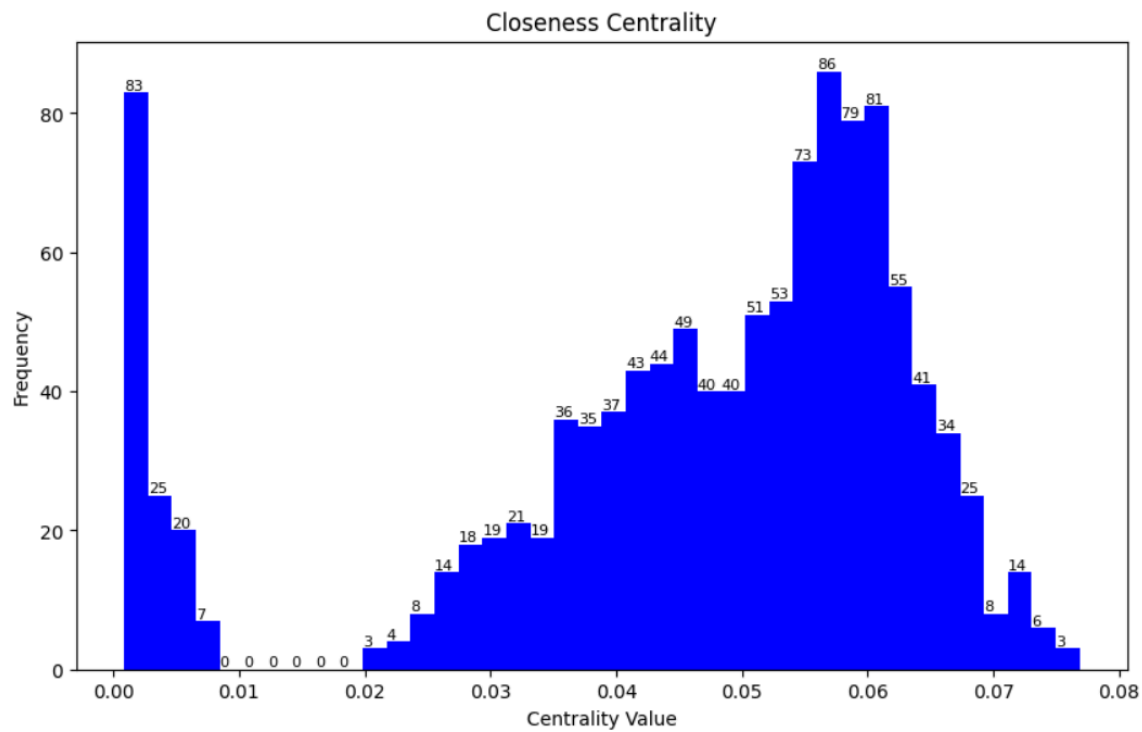
```

# Plot distribution of centrality values for Closeness Centrality
plt.figure(figsize=(10, 6))
plt.hist(list(close_centrality.values()), bins=40, color='b', alpha=0.7) # Increase bin size to 40
plt.title("Closeness Centrality")
plt.xlabel('Centrality Value')
plt.ylabel('Frequency')

# Write frequency on top of each bin
for i in range(40):
    plt.text(plt.hist(list(close_centrality.values()), bins=40, color='b', alpha=0.7)[1][i],
             plt.hist(list(close_centrality.values()), bins=40, color='b', alpha=0.7)[0][i],
             str(int(plt.hist(list(close_centrality.values()), bins=40, color='b', alpha=0.7)[0][i])),
             fontsize=8, ha='left', va='bottom')

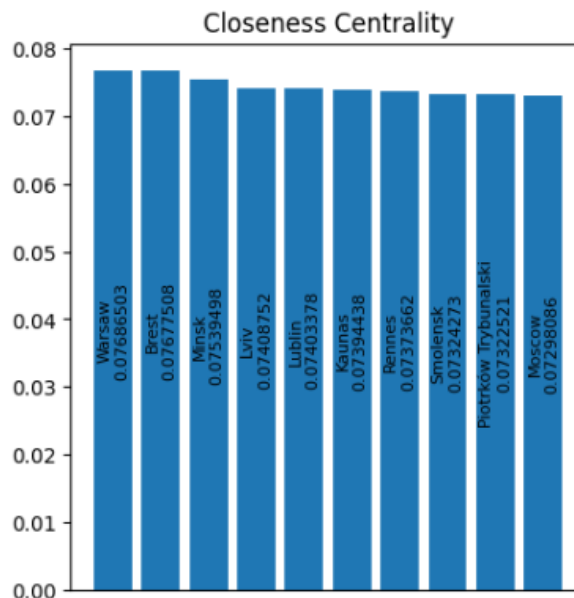
plt.show()

```



Top 10 Description

Nodes	Closeness Centrality
Warsaw	0.076865
Brest	0.076775
Minsk	0.075395
Lviv	0.074088
Lublin	0.074034
Kaunas	0.073944
Rennes	0.073737
Smolensk	0.073243
Piotrkow Trybunalski	0.073225
Moscow	0.072981

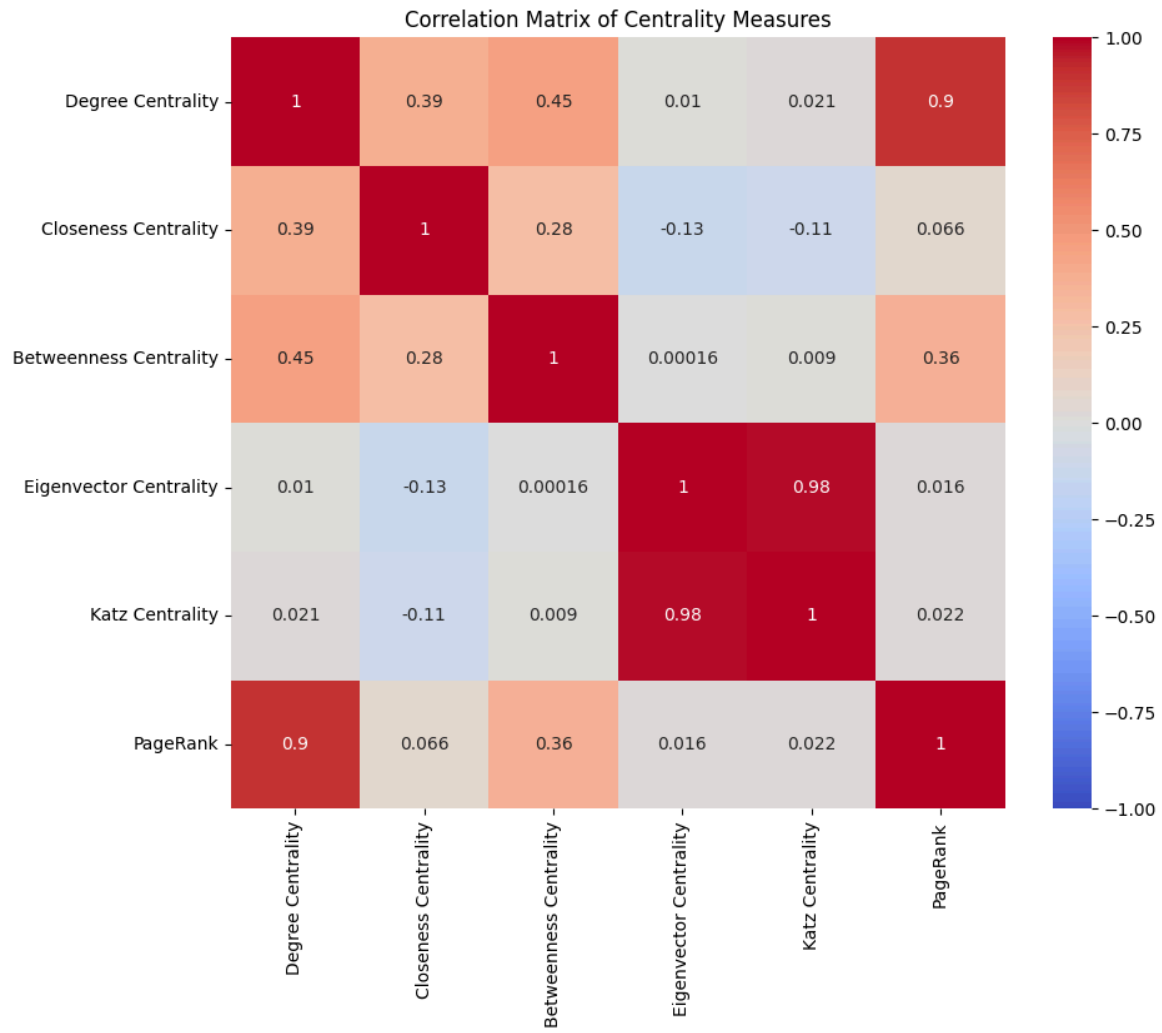


Correlation Matrix Between The Centrality Measures

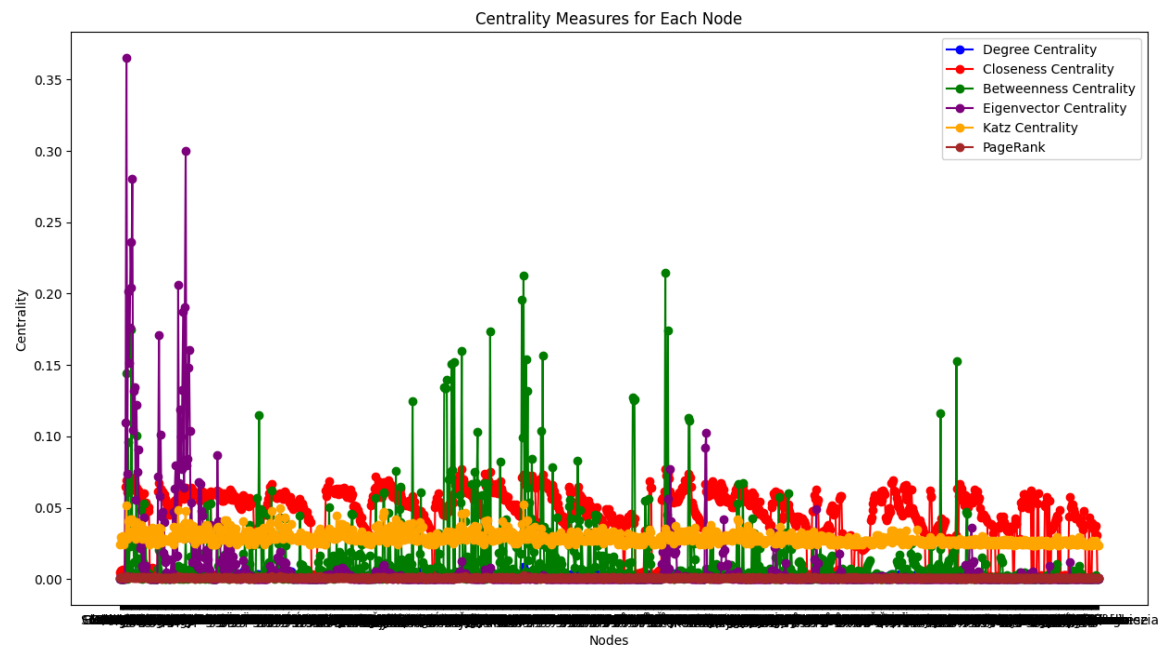
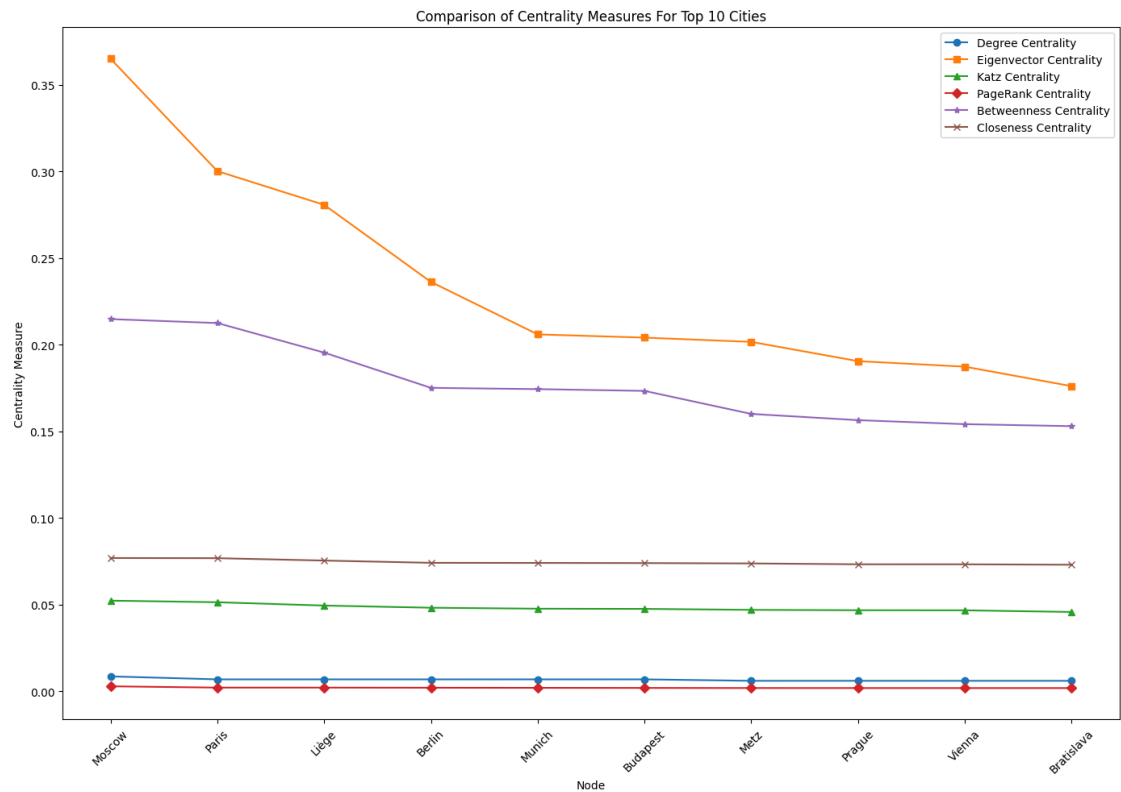
The correlation matrix in the search results shows the correlation between different centrality measures for the European road network. A high correlation between two variables can be beneficial as it indicates a strong relationship. This can simplify analysis and decision-making processes in many cases, as changes in one variable will likely be reflected in the other. For example, in network analysis, a high correlation between centrality measures like eigenvector and Katz centrality can provide more confidence in identifying key nodes that play important roles in the network. This can help target resources or interventions more effectively, as nodes with high centrality in one measure are likely to have high centrality in the other, indicating their critical role in the network's structure and function.

The correlation between degree and PageRank is 0.9, indicating a strong positive correlation between these measures. This means that cities with a high degree of centrality also tend to have a high PageRank centrality, indicating that they are well-connected and linked to other important cities.

However, the correlation between closeness and PageRank is only 0.067, indicating a weak positive correlation between these two measures. This means that cities with a high closeness centrality do not necessarily have a high PageRank centrality, indicating that these two measures capture different aspects of network centrality.



Visualizing Centrality Measures



Clustering Coefficients

Global Clustering Coefficient

```
#Calculate Global Clustering Coefficient
global_clustering_coefficient = nx.transitivity(G)

#Print Global Clustering Coefficient
print("Global Clustering Coefficient:", global_clustering_coefficient)
```

Global Clustering Coefficient: 0.6298424741263426

Local Clustering Coefficient

```
#Calculate Local Clustering Coefficient
local_clustering_coefficient = nx.clustering(G)

#Print each node and it's Clustering Coefficient
for node, local in local_clustering_coefficient.items():
    print(f"Node {node}: Local Clustering Coefficient = {local}")

#Create a table for the top 10 nodes with highest local clustering coefficients
top_10_nodes = sorted(local_clustering_coefficient.items(), key=lambda x: x[1], reverse=True)[:10]
print("Top 10 Nodes with Highest Local Clustering Coefficients:")
print("Node\tLocal Clustering Coefficient")
for node, coeff in top_10_nodes:
    print(f"{node}\t{coeff}")
```

Top 10 Nodes with Highest Local Clustering Coefficients:

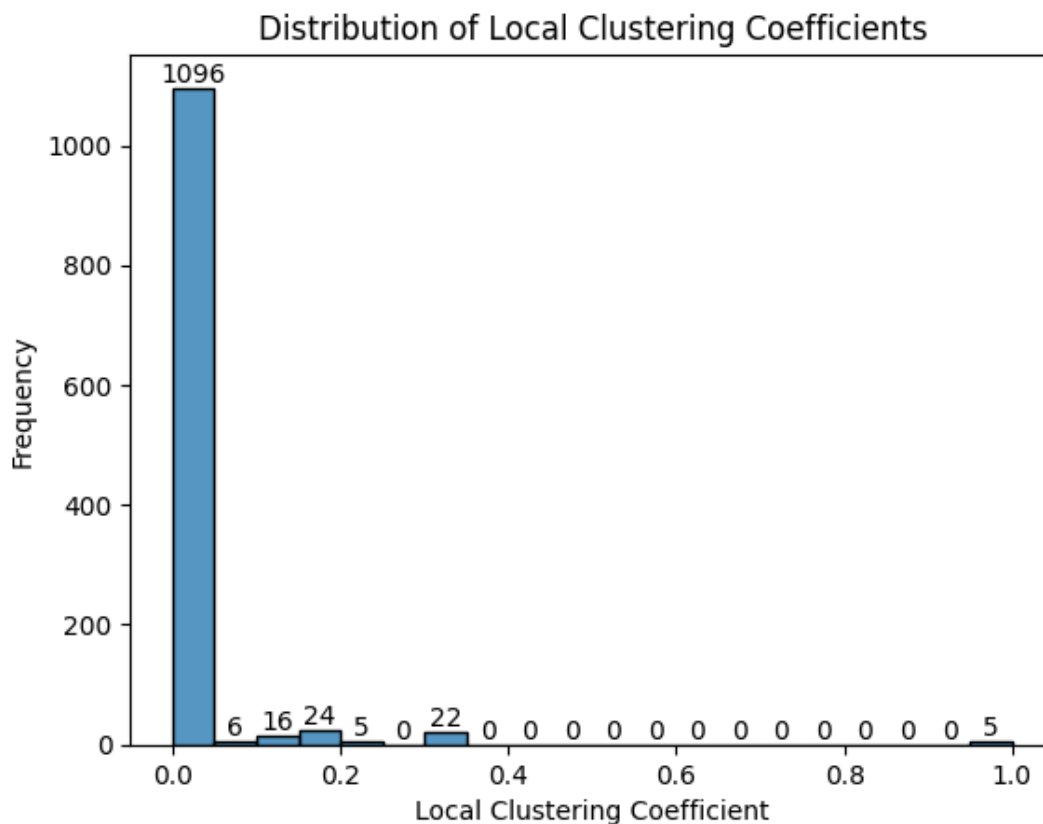
Node	Local Clustering Coefficient
Brindisi	1.0
Domokos	1.0
Vyšné Nemecké	1.0
Postojna	1.0
Spielfeld	1.0
Lausanne	0.3333333333333333
Tortona	0.3333333333333333
Brescia	0.3333333333333333
Ulm	0.3333333333333333
Naples	0.3333333333333333

```
#Create a line chart for the distribution of local clustering coefficients
coefficients = list(local_clustering_coefficient.values())
coefficients.sort()
unique_coefficients = list(set(coefficients))
counts = [[coefficients.count(coeff) for coeff in unique_coefficients]]

#Create a histogram for the distribution of local clustering coefficients
sns.histplot(list(local_clustering_coefficient.values()), bins=20, kde=False)
plt.xlabel('Local Clustering Coefficient')
plt.ylabel('Frequency')
plt.title('Distribution of Local Clustering Coefficients')

#Display the frequency count on top of each bin
for p in plt.gca().patches:
    plt.gca().annotate(str(int(p.get_height())), (p.get_x() + p.get_width() / 2., p.get_height()), ha='center', va='bottom', color='black', fontsize=10)

plt.show()
```



Many nodes have low local clustering coefficients, which suggests that these cities have relatively few connections to their neighbouring cities. This could indicate that these cities are less central in the European road network and may not play as significant a role in transportation and connectivity as other cities with higher local clustering coefficients. However, it is important to note that a low local clustering coefficient does not necessarily mean a city is unimportant in the network. Other centrality measures, such as degree centrality or Katz centrality, may still indicate that the city is well-connected to other cities in the network, even if it is not well-connected to its immediate neighbours.

Average Local Clustering Coefficient

```
#Calculate Average Local Clustering Coefficient
average_local_clustering_coefficient = nx.average_clustering(G)

print("Average Local Clustering Coefficient:", average_local_clustering_coefficient)
```

Average Local Clustering Coefficient: 0.529635811052136

Transitivity and Reciprocity Values

Transitivity

```
transitivity = nx.transitivity(G)
print("Transitivity:", transitivity)
```

Transitivity: 0.0338863395693611

Reciprocity

Reciprocity is a simplified version of transitivity because it considers closed loops of length 2, which can only happen in directed graphs. However, our chosen network, in this case, is undirected. Hence, we cannot find a value for the reciprocity of this network.

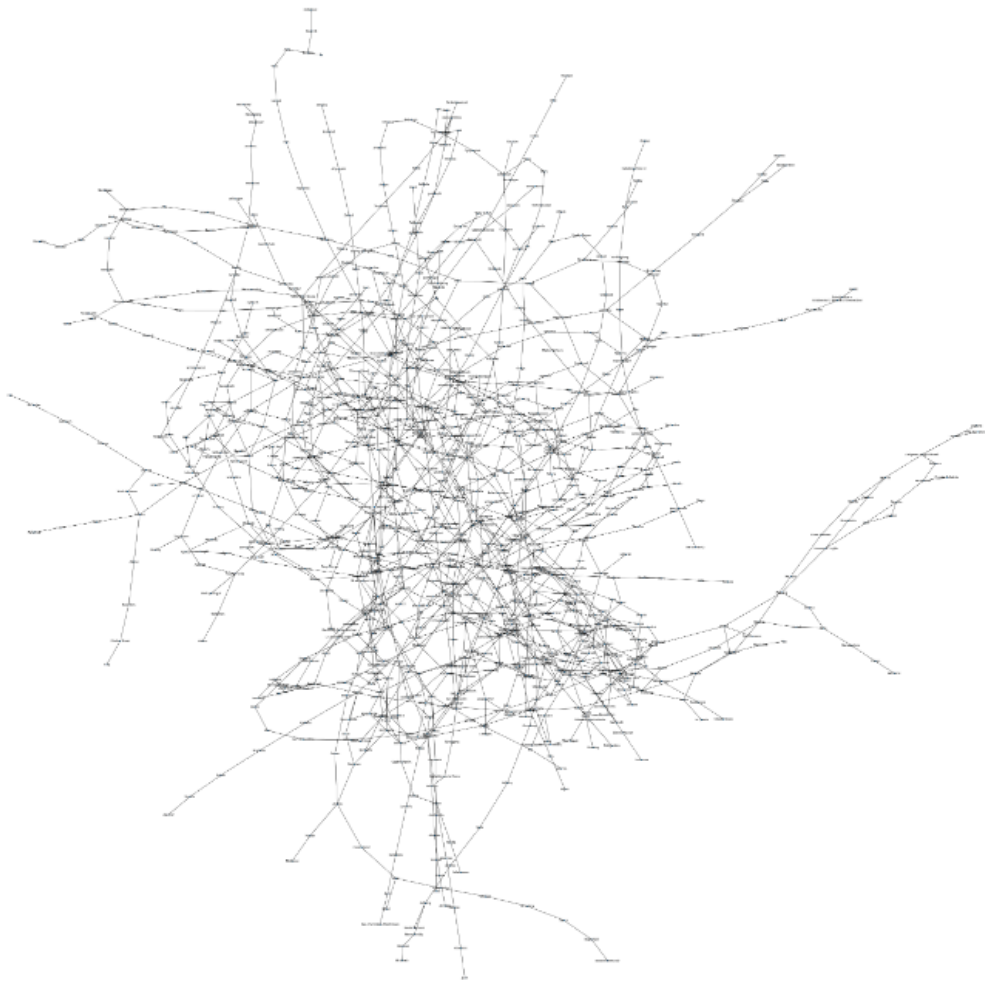
Largest Connected Component

```
#Find the largest connected component
largest_component = max(nx.connected_components(G), key=len)

#Create a subgraph of the largest connected component
largest_component_subgraph = G.subgraph(largest_component)

#Print the number of nodes and edges in the largest connected component
print("Number of nodes in the largest connected component:", largest_component_subgraph.number_of_nodes())
print("Number of edges in the largest connected component:", largest_component_subgraph.number_of_edges())

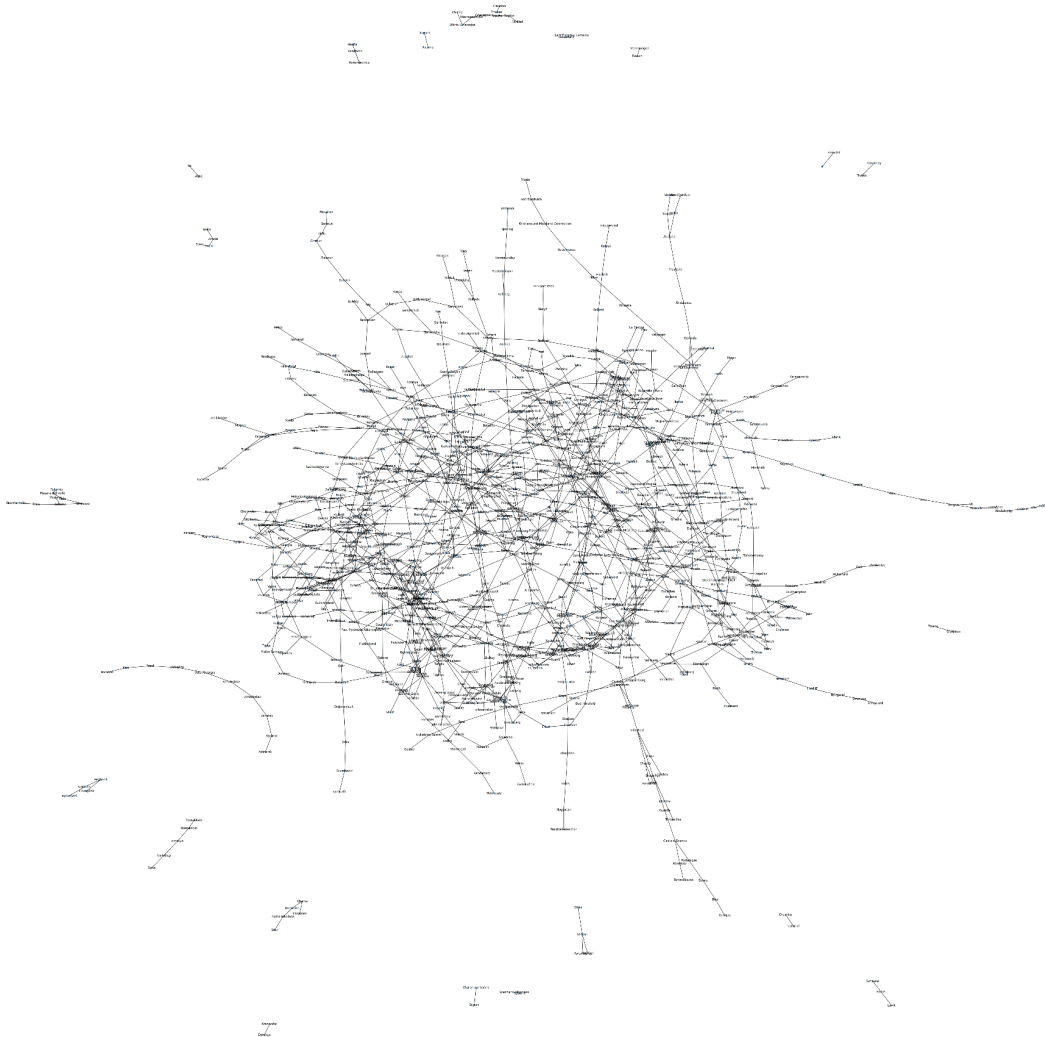
#Visualize the largest connected component
plt.figure(figsize=(18, 12))
nx.draw(largest_component_subgraph, with_labels=False, node_size = 10)
plt.title('Largest Connected Component')
plt.show()
```



Number of nodes in the largest connected component: 1039
Number of edges in the largest connected component: 1305

Complete Network Visualization

```
plt.figure(figsize=(56, 56))  
nx.draw(G, with_labels=True, node_size = 10, font_size = 10)
```



Network 2 - General Relativity and Quantum Cosmology Collaboration Network

This dataset covers the scientific collaborations between authors' papers submitted to the General Relativity and Quantum Cosmology categories. The network we get from the dataset we chose contains nodes representing the authors of the paper and the edges representing the collaborations between the authors.

Overview

In this network, there is an undirected edge from an author to the co-author of a paper.

- If multiple authors write a paper, then the graph between them is a complete subgraph.
 - If there are n authors, then the number of nodes in the subgraph will be n , and the total number of edges in this subgraph will be $n!$

This particular dataset covers papers published between January 1993 and April 2003.

Additional Statistics

Nodes	5242
Edges	14496
Nodes in the largest WCC	4158(0.793)
Edges in the largest WCC	13428(0.926)
Nodes in the largest SCC	4158(0.793)
Edges in the largest SCC	13428(0.926)
Average clustering coefficient	0.5296
Number of triangles	48260
Fraction of closed triangles	0.3619
Diameter(longest shortest path)	17
90-percentile effective diameter	7.6

Network Statistics

Degree Distribution Of This Network

We have used the code below to obtain a degree distribution for our network.

```

#Define the Bin Size and Bins
bin_size = 1
bins = range(min(degrees), max(degrees) + bin_size, bin_size)

#Create Histogram
plt.figure(figsize=(18, 6))
plt.hist(degrees, bins=bins, alpha=0.5, color='blue', edgecolor='black', linewidth=1.5)

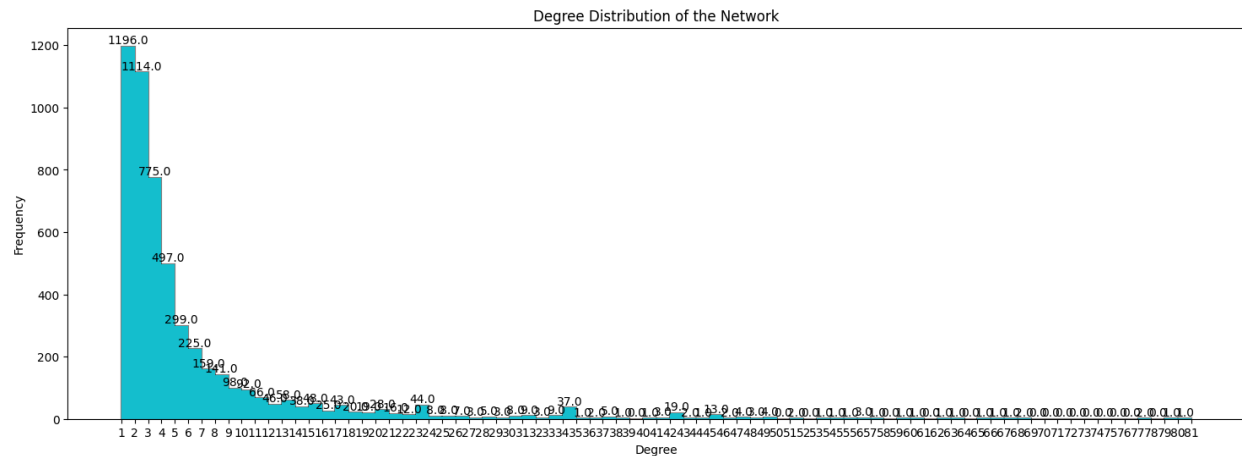
#Add Text Labels for Bin
for i, bin_value in enumerate(bins[:-1]):
    plt.text(bin_value + bin_size/2, plt.hist(degrees, bins=bins)[0][i], str(plt.hist(degrees, bins=bins)[0][i]), ha='center', va='bottom')

#Set x-axis tick locations and labels
plt.xticks(range(min(degrees), max(degrees)+1, 1))

#Set labels and title
plt.xlabel('Degree')
plt.ylabel('Frequency')
plt.title('Degree Distribution of the Network')

#Show plot
plt.show()

```



Additional Statistics

The image below presents our code and the obtained results for additional statistics for our chosen network. Besides the number of nodes and edges, we have also found out the following:

- Maximum Degree of a node in the network.
- Minimum Degree of a node in the network.
- The Average Degree of a node in the network.
- The Standard deviation of the Degree Distribution in the network.

```

print('Nodes:', len(G.nodes))
print('Edges:', len(G.edges))

max_degree = max(degrees)
min_degree = min(degrees)
avg_degree = np.mean(degrees)
std_degree = np.std(degrees)

print("Max Degree of a Node:", max_degree)
print("Min Degree of a Node:", min_degree)
print("Average Degree of Nodes:", avg_degree)
print("Standard Deviation of Degree Distribution:", std_degree)

```

```

Nodes: 5242
Edges: 14496
Max Degree of a Node: 81
Min Degree of a Node: 1
Average Degree of Nodes: 5.530713468141931
Standard Deviation of Degree Distribution: 7.918407301678446

```

Centrality Measures

Degree Centrality

This graph describes the authors and their number of collaborations. The value of the y-axis extends up to 1, starting from 0. The value on the y-axis is very small, which suggests that the number of collaborations is less.

The graph shows that no one has collaborated with all the authors because no node shows the centrality measure 1.

The network size can influence the Degree of centrality because an author who has collaborated with 10 authors in a small network may have higher centrality than an author who has collaborated with 20 authors in a large network.

```

#Calculate Degree Centrality
deg_centrality = nx.degree_centrality(G)

#Print each node and its Degree Centrality
for node, centrality in deg_centrality.items():
    print(f"Node {node}: Degree Centrality = {centrality}")

```

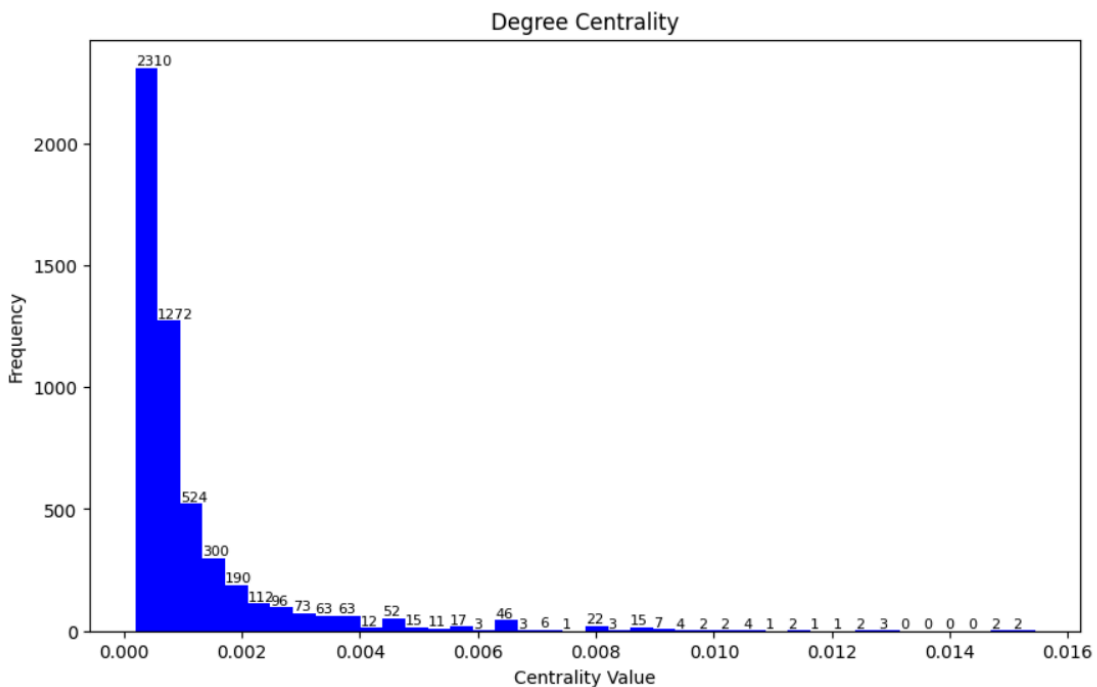
```

# Plot distribution of centrality values for Degree Centrality
plt.figure(figsize=(10, 6))
plt.hist(list(deg Centrality.values()), bins=40, color='b', alpha=0.7) # Increase bin size to 40
plt.title("Degree Centrality")
plt.xlabel('Centrality Value')
plt.ylabel('Frequency')

# Write frequency on top of each bin
for i in range(40):
    plt.text(plt.hist(list(deg Centrality.values()), bins=40, color='b', alpha=0.7)[1][i],
             plt.hist(list(deg Centrality.values()), bins=40, color='b', alpha=0.7)[0][i],
             str(int(plt.hist(list(deg Centrality.values()), bins=40, color='b', alpha=0.7)[0][i])),
             fontsize=8, ha='left', va='bottom')

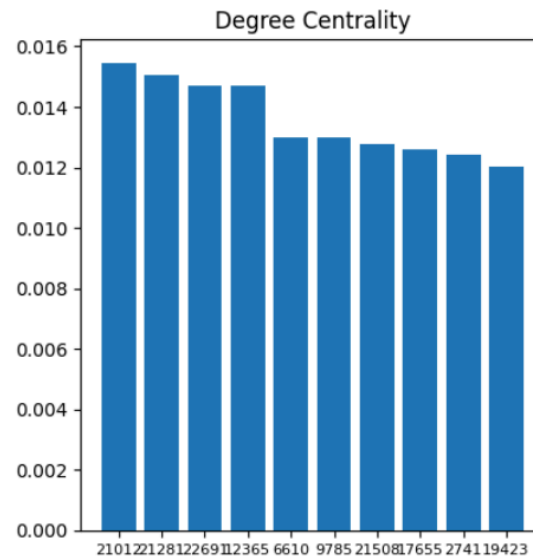
plt.show()

```



Top 10 Description

NODE	Degree Centrality
21012	0.015455
21281	0.015073
22691	0.014692
12365	0.014692
6610	0.012975
9785	0.012975
21508	0.012784
17655	0.012593
2741	0.012402
19423	0.012021



Eigenvector Centrality

Some authors have a high centrality value, representing that they are more influential in research. It is difficult to conclude who the most influential person is from the graph alone, but the nodes with the highest eigenvector centrality could be the most influential authors.

However, using this centrality measure has a few drawbacks, mentioned below:

- This graph won't be able to explain how strong or important the collaboration between 2 authors is for a particular paper.
- Eigenvector centrality is a relative measure because the centrality of a particular node depends on the centrality scores of other authors with whom they collaborate.

```
#Calculate Eigenvector Centrality
eigenvector_centrality = nx.eigenvector_centrality(G)

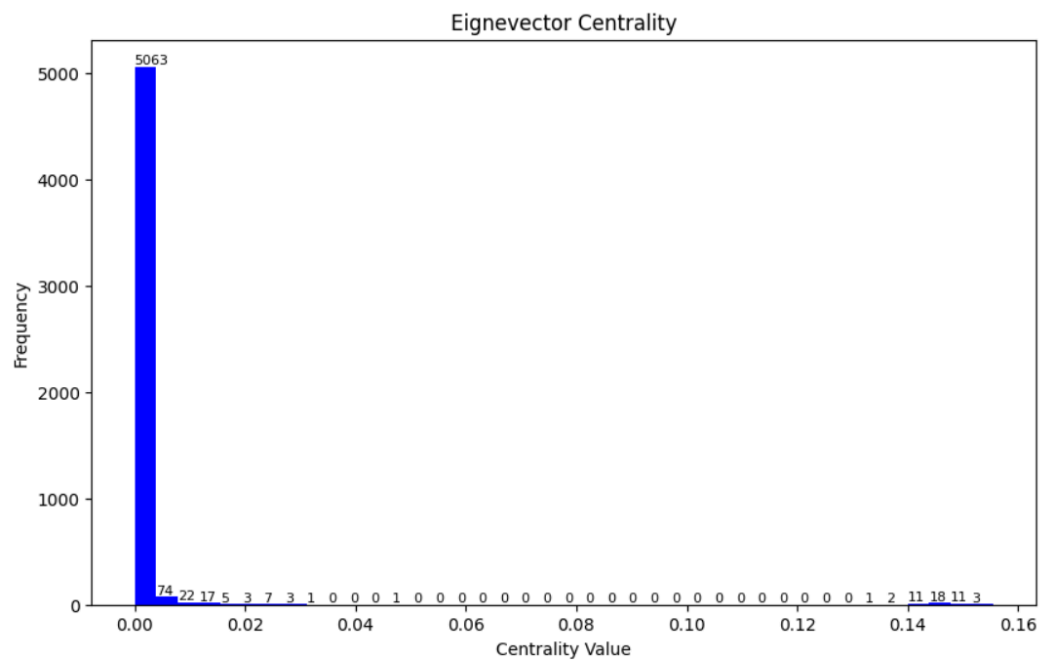
#Print each node and its Eigenvector Centrality
for node, centrality in eigenvector_centrality.items():
    print(f"Node {node}: Eigenvector Centrality = {centrality}")
```



```
# Plot distribution of centrality values for Eigenvector Centrality
plt.figure(figsize=(10, 6))
plt.hist(list(eigenvector Centrality.values()), bins=40, color='b', alpha=0.7) # Increase bin size to 40
plt.title("Eigenvector Centrality")
plt.xlabel('Centrality Value')
plt.ylabel('Frequency')

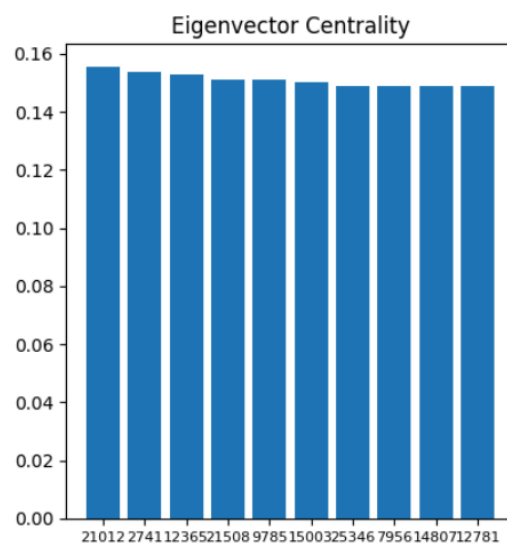
# Write frequency on top of each bin
for i in range(40):
    plt.text(plt.hist(list(eigenvector Centrality.values()), bins=40, color='b', alpha=0.7)[1][i],
             plt.hist(list(eigenvector Centrality.values()), bins=40, color='b', alpha=0.7)[0][i],
             str(int(plt.hist(list(eigenvector Centrality.values()), bins=40, color='b', alpha=0.7)[0][i])),
             fontsize=8, ha='left', va='bottom')

plt.show()
```



Top 10 Description

NODE	EigenVector Centrality
21012	0.155561
2741	0.153574
12365	0.153072
21508	0.151194
9785	0.150903
15003	0.150407
25346	0.149069
7956	0.149066
14807	0.149007
12781	0.148881



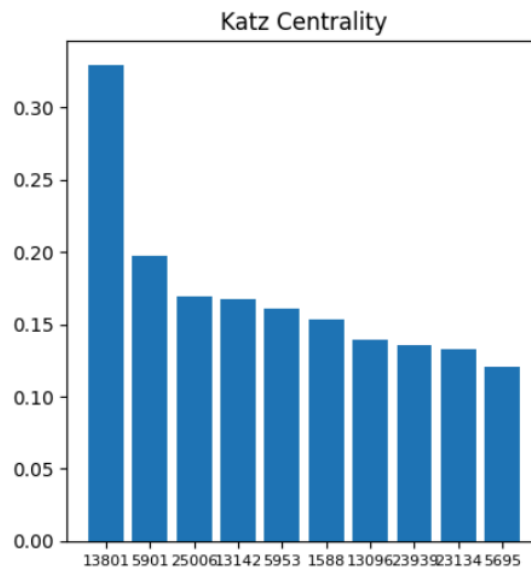
Katz centrality generally infers the potential influence of a node in the network based on the length of the paths connecting to other nodes. An author with many short paths leading to them is likely to be a greater influence when compared to one with fewer or long paths. Katz Centrality can also be used to identify the key players in spreading information or ideas through the collaboration network

Katz Centrality

Centrality Value	Frequency
-0.08	2
-0.07	0
-0.06	1
-0.05	7
-0.04	0
-0.03	1
-0.02	1
-0.01	10
0.00	27
0.01	105
0.02	4282
0.03	517
0.04	127
0.05	68
0.06	30
0.07	17
0.08	22
0.09	4
0.10	5
0.11	3
0.12	2
0.13	1
0.14	3
0.15	1
0.16	1
0.17	2
0.18	0
0.19	1
0.20	0
0.21	0
0.22	0
0.23	0
0.24	0
0.25	0
0.26	0
0.27	0
0.28	0
0.29	0
0.30	0
0.31	0
0.32	1

Top 10 Description

NODE	Katz Centrality
13801	0.329264
5901	0.196795
25006	0.168997
13142	0.167177
5953	0.161180
1588	0.153601
13096	0.139432
23939	0.135106
23134	0.132340
5695	0.120516



PageRank Centrality

Authors with a high PageRank Centrality value will likely get cited by other well-regarded authors. It reflects how much authors' work is referenced by others in the field, suggesting the reference and importance of their research. This helps identify prominent authors, can also be useful in reflecting how impactful the author is and finally helps identify emerging scholars.

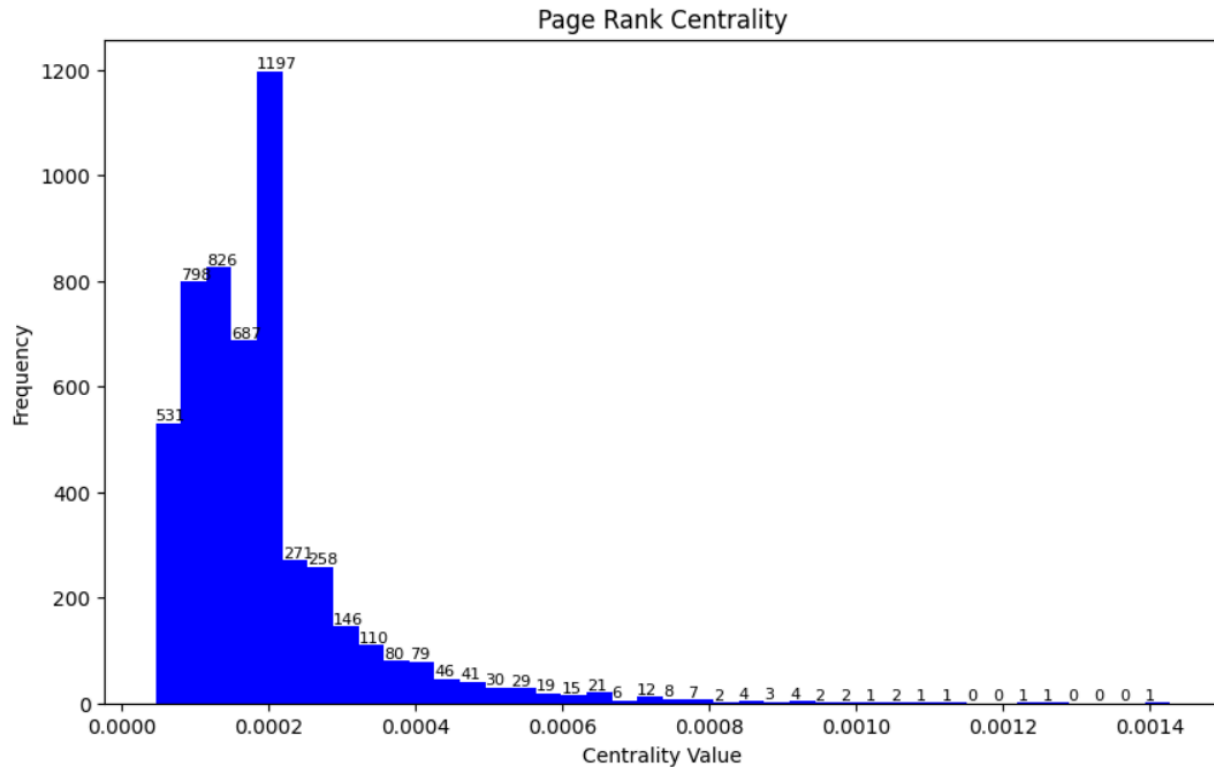
```
pr = nx.pagerank(G, alpha=0.8)

# Print each node and its Page Rank
for node, centrality in pr.items():
    print(f"Node {node}: Page Rank = {centrality}")

# Plot distribution of centrality values for Page Rank Centrality
plt.figure(figsize=(10, 6))
plt.hist(list(pr.values()), bins=40, color='b', alpha=0.7) # Increase bin size to 40
plt.title("Page Rank Centrality")
plt.xlabel('Centrality Value')
plt.ylabel('Frequency')

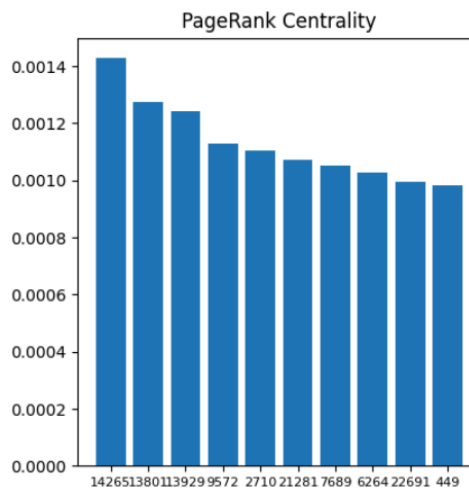
# Write frequency on top of each bin
for i in range(40):
    plt.text(plt.hist(list(pr.values()), bins=40, color='b', alpha=0.7)[1][i],
             plt.hist(list(pr.values()), bins=40, color='b', alpha=0.7)[0][i],
             str(int(plt.hist(list(pr.values()), bins=40, color='b', alpha=0.7)[0][i])),
             fontsize=8, ha='left', va='bottom')

plt.show()
```



Top 10 Description

NODE	PageRank Centrality
14265	0.001428
13801	0.001277
13929	0.001242
9572	0.001130
2710	0.001103
21281	0.001071
7689	0.001053
6264	0.001026
22691	0.000994
449	0.000982



Betweenness Centrality

Betweenness here generally focuses on knowledge flow. Authors with high betweenness centrality might play a crucial role in facilitating the flow of knowledge and ideas between the authors. Authors with high betweenness centrality values will likely be more comfortable working across disciplinary boundaries.

```

bet Centrality = nx.betweenness Centrality(G, normalized = True, endpoints = False)

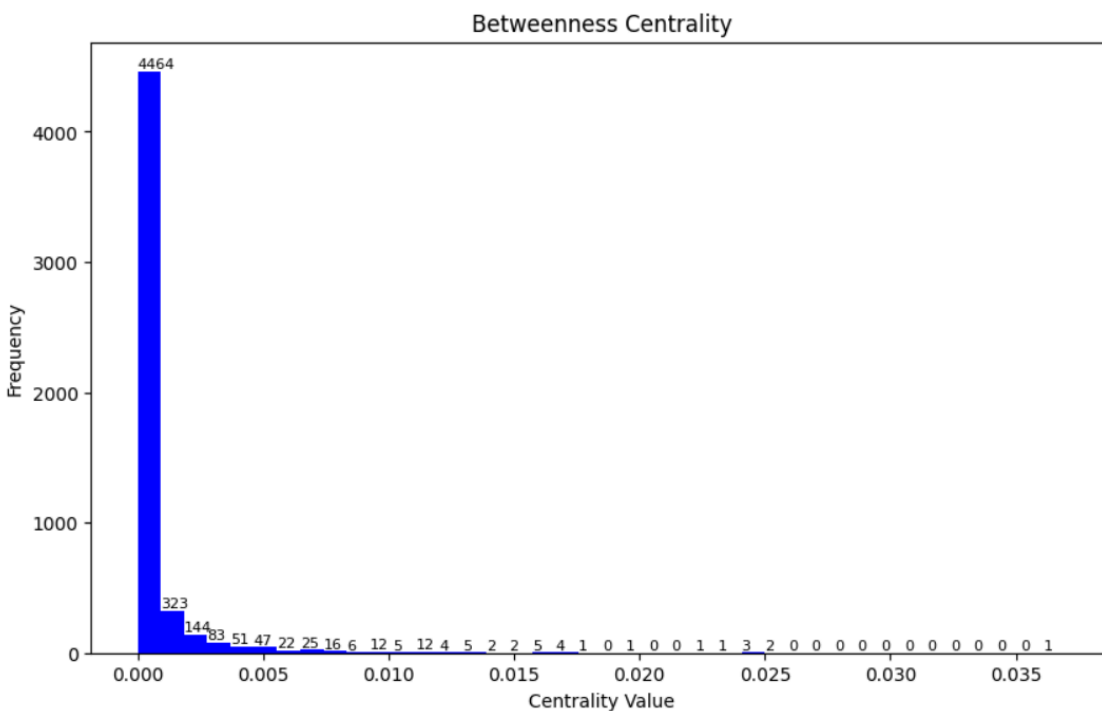
#Print each node and its Betweenness Centrality (excluding nodes with centrality = 0)
for node, centrality in bet Centrality.items():
    if centrality != 0:
        print(f"Node {node}: Betweenness Centrality = {centrality}")

# Plot distribution of centrality values for Betweenness Centrality
plt.figure(figsize=(10, 6))
plt.hist(list(bet Centrality.values()), bins=40, color='b', alpha=0.7) # Increase bin size to 40
plt.title("Betweenness Centrality")
plt.xlabel('Centrality Value')
plt.ylabel('Frequency')

# Write frequency on top of each bin
for i in range(40):
    plt.text(plt.hist(list(bet Centrality.values()), bins=40, color='b', alpha=0.7)[1][i],
             plt.hist(list(bet Centrality.values()), bins=40, color='b', alpha=0.7)[0][i],
             str(int(plt.hist(list(bet Centrality.values()), bins=40, color='b', alpha=0.7)[0][i])),
             fontsize=8, ha='left', va='bottom')

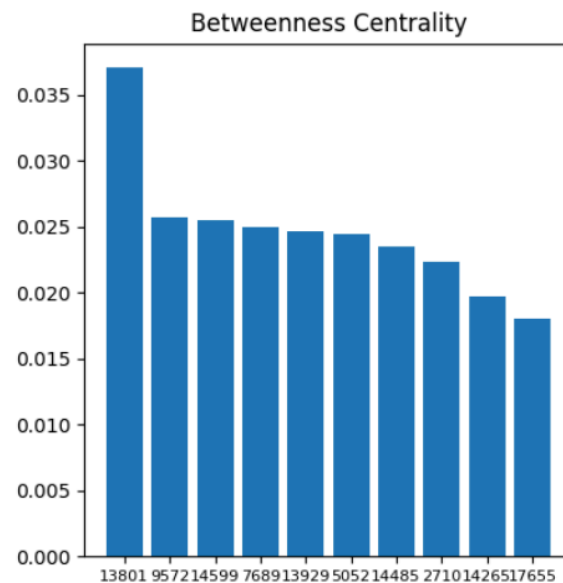
plt.show()

```



Top 10 Description

NODE	Betweenness Centrality
13801	0.037027
9572	0.025689
14599	0.025488
7689	0.024971
13929	0.024653
5052	0.024422
14485	0.023503
2710	0.022355
14265	0.019731
17655	0.017999



Closeness Centrality

The higher closeness centrality value shows that those authors are closer to all other authors in the network. Authors with high-value scores will likely be more central to the collaboration network. This particularly focuses on how easily reachable an author is from all other authors. A high closeness centrality indicates that, on average, an author is close to most other authors in the network

```
#Calculate Closeness centrality
close_centrality = nx.closeness_centrality(G)

#Print each node and its Closeness centrality
for node, centrality in close_centrality.items():
    print(f"Node {node}: Closeness centrality = {centrality}")

# Plot distribution of centrality values for Closeness Centrality
plt.figure(figsize=(10, 6))
plt.hist(list(close_centrality.values()), bins=40, color='b', alpha=0.7) # Increase bin size to 40
plt.title("Closeness Centrality")
plt.xlabel('Centrality Value')
plt.ylabel('Frequency')

# Write frequency on top of each bin
for i in range(40):
    plt.text(plt.hist(list(close_centrality.values()), bins=40, color='b', alpha=0.7)[1][i],
             plt.hist(list(close_centrality.values()), bins=40, color='b', alpha=0.7)[0][i],
             str(int(plt.hist(list(close_centrality.values()), bins=40, color='b', alpha=0.7)[0][i])),
             fontsize=8, ha='left', va='bottom')

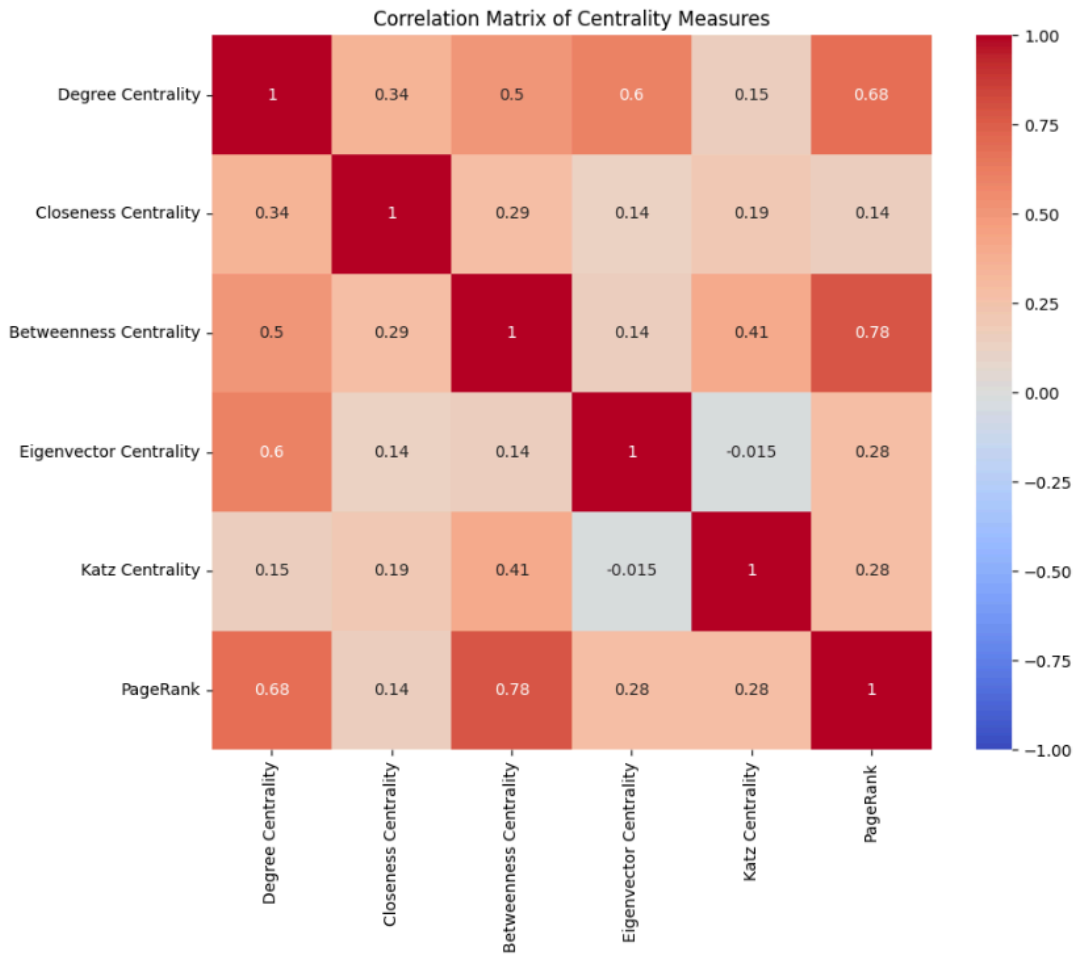
plt.show()
```


(PageRank) also tend to lie on many shortest paths between other nodes (Betweenness Centrality).

The negative value for eigenvector and Katz centrality suggests that these measures are insignificant in this network. Eigenvector centrality measures a node's influence based on its neighbours' influence. In contrast, Katz centrality measures a node's influence based on its neighbours' number and influence. The negative value indicates that these measures are not useful in identifying influential authors in this network.

On the other hand, the positive values for betweenness centrality and PageRank suggest that these measures are significant in this network. Betweenness centrality measures the extent to which a node lies on the shortest paths between other nodes, indicating its importance as a bridge or mediator in the network. PageRank is a measure of the importance of a node based on the number and quality of other nodes linking to it, indicating its overall importance in the network.

The degree centrality and PageRank scores of 0.68 suggest a moderate correlation between these two measures. Degree centrality measures the number of direct connections a node has, while PageRank measures a node's overall importance based on its neighbours' importance. A moderate correlation suggests that authors with many direct connections also have important neighbors, indicating their overall importance in the network.



Visualizing Centrality Measures



Clustering Coefficients

Global Clustering Coefficient

```
#Calculate Global Clustering Coefficient
global_clustering_coefficient = nx.transitivity(G)

#Print Global Clustering Coefficient
print("Global Clustering Coefficient:", global_clustering_coefficient)
```

Global Clustering Coefficient: 0.6298424741263426

Local Clustering Coefficient

```
#Calculate Local Clustering Coefficient
local_clustering_coefficient = nx.clustering(G)

#Print each node and it's Clustering Coefficient
for node, local in local_clustering_coefficient.items():
    print(f"Node {node}: Local Clustering Coefficient = {local}")
```

```
#Count the number of nodes with local clustering coefficient equal to 1
nodes_with_coefficient_1 = sum(1 for coeff in local_clustering_coefficient.values() if coeff == 1)
print(f"Number of Nodes with Local Clustering Coefficient 1: {nodes_with_coefficient_1}")
```

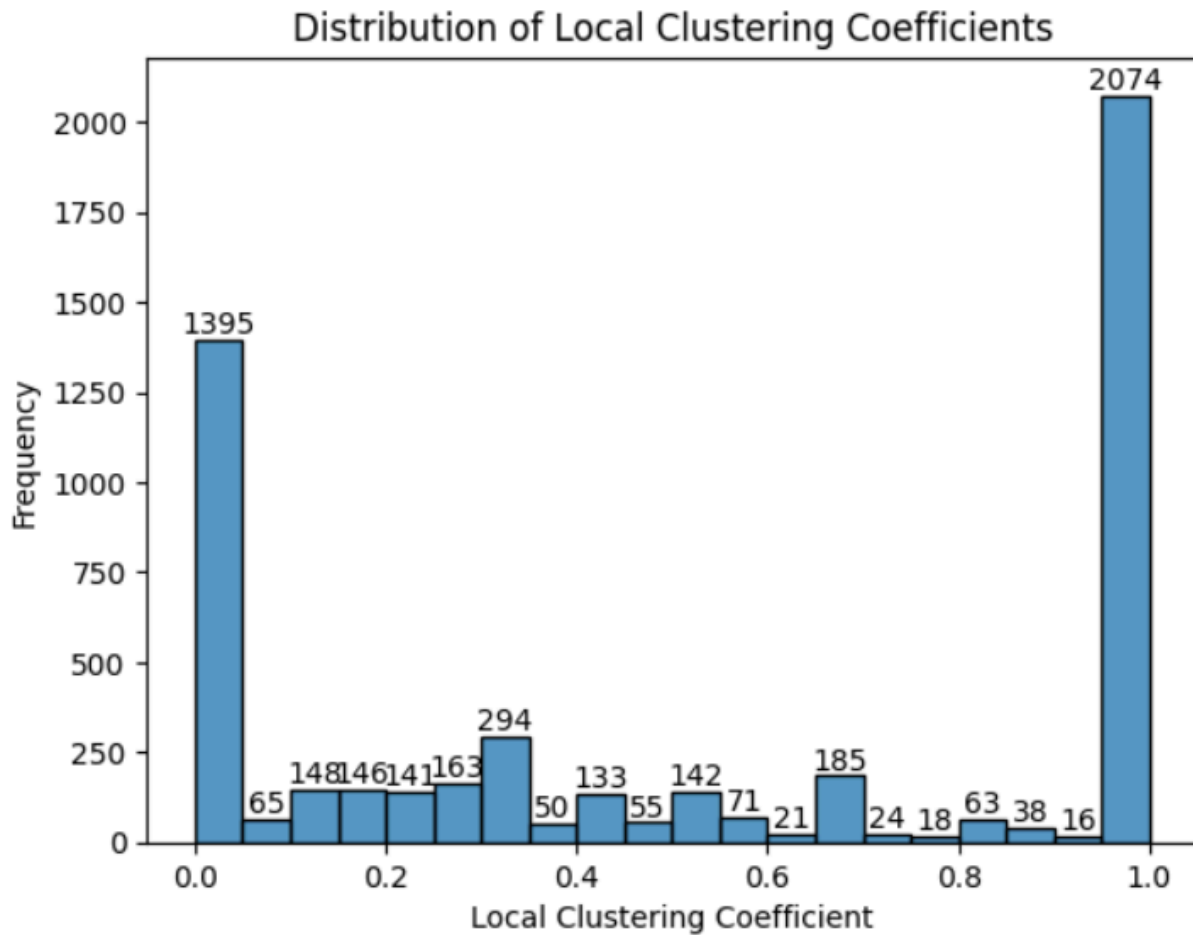
Number of Nodes with Local Clustering Coefficient 1: 2038

```
#Create a line chart for the distribution of local clustering coefficients
coefficients = list(local_clustering_coefficient.values())
coefficients.sort()
unique_coefficients = list(set(coefficients))
counts = [coefficients.count(coeff) for coeff in unique_coefficients]

#Create a histogram for the distribution of local clustering coefficients
sns.histplot(list(local_clustering_coefficient.values()), bins=20, kde=False)
plt.xlabel('Local Clustering Coefficient')
plt.ylabel('Frequency')
plt.title('Distribution of Local Clustering Coefficients')

#Display the frequency count on top of each bin
for p in plt.gca().patches:
    plt.gca().annotate(str(int(p.get_height())), (p.get_x() + p.get_width() / 2., p.get_height()), ha='center', va='bottom', color='black', fontsize=10)

plt.show()
```



Average Local Clustering Coefficient

```
#Calculate Average Local Clustering Coefficient
average_local_clustering_coefficient = nx.average_clustering(G)

print("Average Local Clustering Coefficient:", average_local_clustering_coefficient)
```

Average Local Clustering Coefficient: 0.529635811052136

Transitivity and Reciprocity

Transitivity

```
transitivity = nx.transitivity(G)
print("Transitivity:", transitivity)
```

Transitivity: 0.6298424741263426

Reciprocity

Reciprocity is a simplified version of transitivity because it considers closed loops of length 2, which can only happen in directed graphs. However, our chosen network, in this case, is undirected. Hence, we cannot find a value for the reciprocity of this network.

Largest Connected Component

```
#Find the largest connected component
largest_component = max(nx.connected_components(G), key=len)

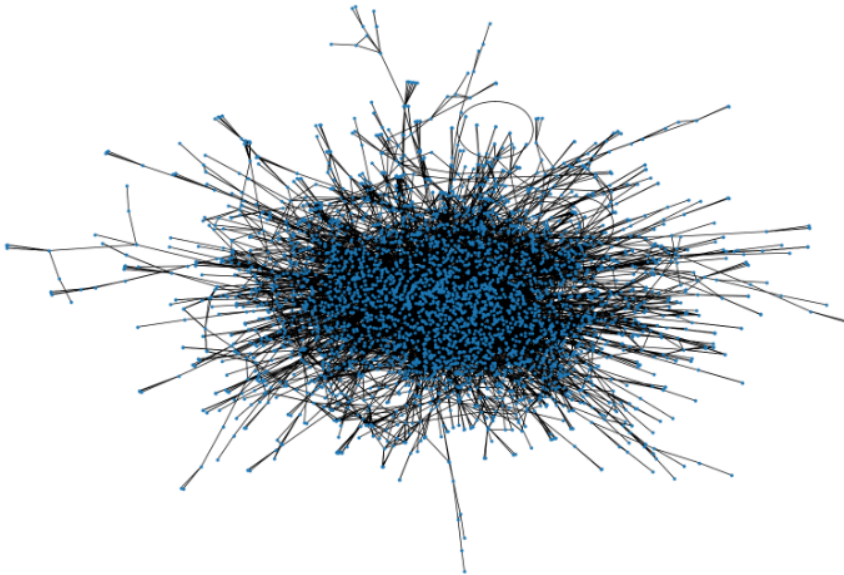
#Create a subgraph of the largest connected component
largest_component_subgraph = G.subgraph(largest_component)

#Print the number of nodes and edges in the largest connected component
print("Number of nodes in the largest connected component:", largest_component_subgraph.number_of_nodes())
print("Number of edges in the largest connected component:", largest_component_subgraph.number_of_edges())

#Visualize the largest connected component
plt.figure(figsize=(18, 12))
nx.draw(largest_component_subgraph, with_labels=False, node_size = 10)
plt.title('Largest Connected Component')
plt.show()
```

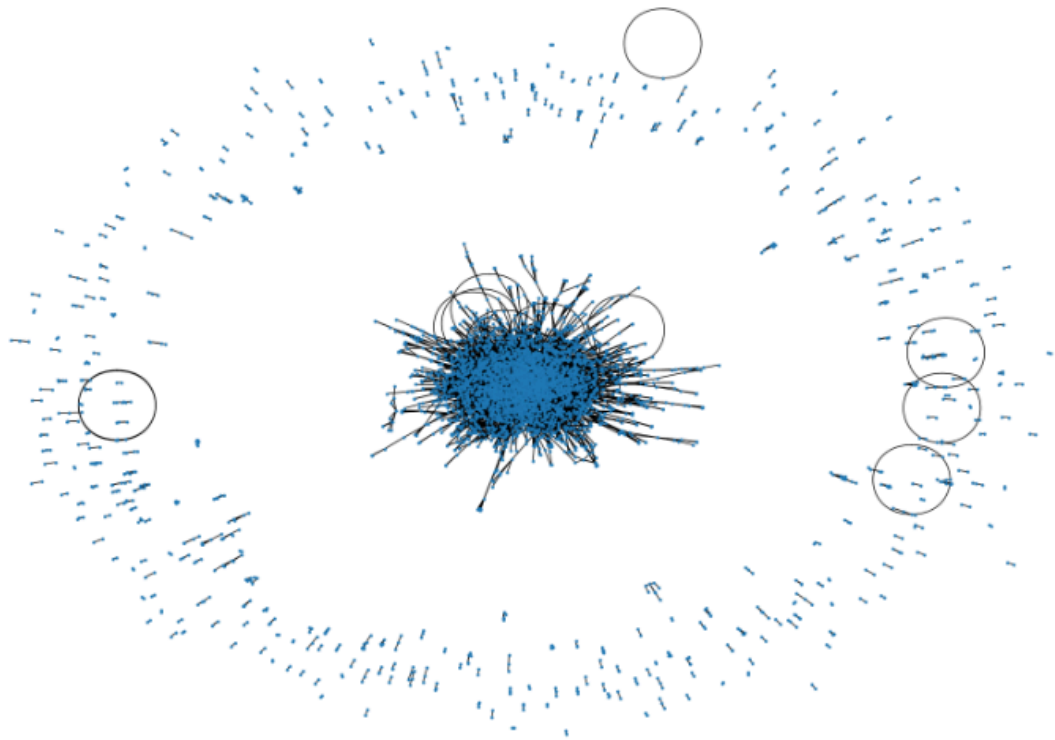
```
Number of nodes in the largest connected component: 4158
Number of edges in the largest connected component: 13428
```

Largest Connected Component



Network Visualization

```
plt.figure(figsize=(18, 12))  
nx.draw(G, with_labels=False, node_size = 5, width = 1)  
plt.show()
```



Comparative Analysis Between The Two Networks

Centrality Measures From Network - 1

```
#Compare centrality measures based on criteria

mean_degree = sum(deg_centrality.values()) / len(deg_centrality)
mean_closeness = sum(close_centrality.values()) / len(close_centrality)
mean_betweenness = sum(bet_centrality.values()) / len(bet_centrality)
mean_eigenvector = sum(eigenvector_centrality.values()) / len(eigenvector_centrality)
mean_katz = sum(katz_centrality.values()) / len(katz_centrality)
mean_pagerank = sum(pr.values()) / len(pr)

print("Mean Degree Centrality:", mean_degree)
print("Mean Closeness Centrality:", mean_closeness)
print("Mean Betweenness Centrality:", mean_betweenness)
print("Mean Eigenvector Centrality:", mean_eigenvector)
print("Mean Katz Centrality:", mean_katz)
print("Mean PageRank:", mean_pagerank)
```

Mean Degree Centrality: 0.002057944872638308
Mean Closeness Centrality: 0.04566886623059432
Mean Betweenness Centrality: 0.011629407659036898
Mean Eigenvector Centrality: 0.007259108836993484
Mean Katz Centrality: 0.028890194343833876
Mean PageRank: 0.0008517887563884146

Centrality Measures From Network - 2

```
#Compare centrality measures based on criteria

mean_degree = sum(deg_centrality.values()) / len(deg_centrality)
mean_closeness = sum(close_centrality.values()) / len(close_centrality)
mean_betweenness = sum(bet_centrality.values()) / len(bet_centrality)
mean_eigenvector = sum(eigenvector_centrality.values()) / len(eigenvector_centrality)
mean_katz = sum(katz_centrality.values()) / len(katz_centrality)
mean_pagerank = sum(pr.values()) / len(pr)

print("Mean Degree Centrality:", mean_degree)
print("Mean Closeness Centrality:", mean_closeness)
print("Mean Betweenness Centrality:", mean_betweenness)
print("Mean Eigenvector Centrality:", mean_eigenvector)
print("Mean Katz Centrality:", mean_katz)
print("Mean PageRank:", mean_pagerank)
```

Mean Degree Centrality: 0.001055278280507871
Mean Closeness Centrality: 0.10630130466095028
Mean Betweenness Centrality: 0.0006062707858585091
Mean Eigenvector Centrality: 0.0016145570004595063
Mean Katz Centrality: 0.001991474673439358
Mean PageRank: 0.0001907668828691324

Inferences using various centrality measures from both networks

Degree Centrality

The mean degree centrality of network 1 is 0.0021, while that of network 2 is 0.0011. This suggests that nodes are, on average, more connected in network 1.

Closeness Centrality

The mean closeness centrality of network 1 is 0.0457, while that of network 2 is 0.1063. This suggests that nodes are much closer to each other on average in network 1, indicating a more tightly connected network.

Betweenness Centrality

The mean betweenness centrality of network 1 is 0.0116, while that of network 2 is 0.0006. This suggests that nodes in network 1 play a more critical role in connecting other nodes.

Eigenvector Centrality

The mean eigenvector centrality of 0.0073 suggests that nodes in network 1 have, on average, moderate influence or importance within the network. The mean eigenvector centrality of 0.0016 in network 2 indicates that nodes have a lower influence or importance than network 1.

Katz Centrality

The mean Katz centrality of 0.0289 suggests that nodes in network 1 have, on average, moderate influence or importance considering both direct and indirect connections. network 2: The mean Katz centrality of 0.0020 in network 2 indicates that nodes have a lower influence or importance than network 1.

PageRank

The higher mean PageRank of 0.0009 suggests that nodes in network 1 are, on average, more important or influential according to the PageRank algorithm. The lower mean PageRank of 0.0002 in network 2 indicates that nodes are less essential or influential according to the PageRank algorithm compared to network 1.

Conclusion

In conclusion, network 1 (Euroroads) has a more tightly connected network with nodes with more connections and closer proximity, playing a more critical role in connecting other nodes. Nodes in network 1 also have more influence or importance within the network, considering both direct and indirect connections and the PageRank algorithm. Therefore, network 1 has a more complex and interconnected structure than network 2 (General Relativity and Quantum Cosmology Collaboration Network).

If you wish to check the code for the project, refer to the following GitHub link:

<https://github.com/RonitGupta2002/SNA-Project-Network-Analysis>